

Software Requirements Specifications (SRS)

BugBoard26

Ver. 1.0

Gruppo di Progetto: ID25

Luca Santangelo

N86002450

Manuel Montuori

N86001690

14 luglio 2026

Indice

1	Introduzione	6
1.1	Scopo del documento	6
1.2	Campo di applicazione del sistema	6
1.3	Definizioni, sinonimi adottati e abbreviazioni	7
1.3.1	Definizioni (Glossario)	7
1.3.2	Sinonimi adottati	9
1.3.3	Abbreviazioni	9
1.4	Riferimenti	9
1.5	Panoramica del documento	9
2	Descrizione generale	12
2.1	Prospettiva del prodotto	12
2.2	Funzionalità del prodotto	12
2.3	Utenti e attori del sistema	13
3	Requisiti specifici	14
3.1	Requisiti funzionali	14
3.1.1	RE01 - Log in utente	14
3.1.2	RE02 - Nuova utenza (Amministratore)	14
3.1.3	RE03 - Segnala issue	15
3.1.4	RE04 - Assegna issue	16
3.1.5	RE05 - Visualizza riepilogo issue	17
3.1.6	RE06 - Visualizza report team	18
3.1.7	RE07 - Utente visualizza le proprie issue assegnate	18
3.1.8	RE08 - Filtra issue	18
3.1.9	RE09 - Ordina issue	19
3.1.10	RE10 - Ricerca issue	20
3.1.11	RE11 - Visualizza notifiche	21
3.1.12	RE12 - Lettura notifiche	21
3.1.13	RE13 - Workload	22
3.2	Modellazione dei casi d'uso tramite Use Case Diagram	22
3.2.1	Use Case Diagram	22
3.2.2	Use Case Diagram parziale 1	23
3.2.3	Use Case Diagram parziale 2	24
3.2.4	Use Case Diagram parziale 3	25
3.2.5	Use Case Diagram parziale 4	26
3.2.6	Use Case Diagram parziale 6	27

3.3	Requisiti non funzionali	27
3.3.1	RNF01 - Sicurezza	27
3.3.2	RNF02 - Affidabilità	28
3.3.3	RNF03 - Prestazioni	28
3.3.4	RNF04 - Usabilità	28
3.3.5	RNF05 - Scalabilità	28
3.3.6	RNF06 - Portabilità e indipendenza tra i componenti	28
3.3.7	RNF07 - Manutenibilità	29
3.4	Formalizzazione di casi d'uso significativi	29
3.4.1	Tabella di Cockburn 1	29
3.4.2	Tabella di Cockburn 2	30
3.5	Prototipazione visuale del caso d'uso 3.3.3 Segnala Issue	31
3.5.1	Segnala Issue - Mockup 1	32
3.5.2	Segnala Issue - Mockup 2	33
3.5.3	Segnala Issue - Mockup 3	34
3.5.4	Segnala Issue - Mockup 4	35
3.6	Prototipazione visuale del caso d'uso Assegna Issue	35
3.6.1	Assegna Issue - Mockup 1	36
3.6.2	Assegna Issue - Mockup 2	37
3.6.3	Assegna Issue - Mockup 3	38
3.6.4	Assegna Issue - Mockup 4	39
4	Documento di design del sistema	40
4.1	Analisi dell'Architettura	40
4.1.1	Server	40
4.1.2	Persistenza	41
4.1.3	Client	41
4.2	Scelte tecnologiche e motivazioni	42
4.2.1	Server	42
4.2.2	Persistenza	42
4.2.3	Client	43
5	Documento di Design del Software	44
5.1	Descrizione dello schema di persistenza dei dati	44
5.2	Class Diagram di Design	44
5.2.1	Class Diagram 1- Model	44
5.2.2	Class Diagram 2- Funzionalità 2	46
5.2.3	Class Diagram 3- Funzionalità 3	46
5.2.4	Class Diagram 4- Funzionalità 4 e 14	47
5.2.5	Class Diagram 5- Funzionalità 11	48

5.2.6	Class Diagram 6- Funzionalità 17	49
5.3	Package Diagram Frontend	51
5.4	Descrizione e motivazione delle scelte di software design adottate	52
5.5	Strumento di Versioning	53
5.6	Containerizzazione, build automatica e Cloud Deploy	54
5.6.1	Build automatica	54
5.6.2	Containerizzazione	54
5.6.3	Deploy AWS	56
5.6.4	Report di qualità del codice	56
6	Test Plan	59
6.1	Test della funzionalità RE04 – Assegna issue	59
6.1.1	Analisi delle classi di equivalenza	59
6.1.2	TC1 – Assegnazione corretta	59
6.1.3	TC2 – Utente assegnatario inesistente	60
6.1.4	TC3 – Issue inesistente	60
6.1.5	TC4 – Issue inesistente e utente inesistente	60
6.1.6	TC5 – Verifica creazione notifica	60
6.1.7	TC6 – Visualizzazione dei bug assegnati	61
6.1.8	TC7 – Assegnazione tentata da utente non amministratore	61
6.1.9	Copertura delle classi di equivalenza	61
6.2	Test del metodo assignIssue – IssueWriteService	61
6.2.1	Metodo sottoposto a test	61
6.2.2	Strategia adottata	62
6.2.3	Black box	62
6.2.4	Casi di test progettati	62
6.2.5	White box	63
6.2.6	Oracle dei test	64
6.3	Test del metodo createNotification – NotificationWriteService	64
6.3.1	Metodo sottoposto a test	64
6.3.2	Strategia adottata	65
6.3.3	Black box	65
6.3.4	Casi di test progettati	65
6.3.5	Oracle dei test	66
6.4	Test del metodo getMonthlyReport – UserReadService	67
6.4.1	Metodo sottoposto a test	67
6.4.2	Strategia adottata	67
6.4.3	Black box	67
6.4.4	Casi di test progettati	68

6.4.5	Oracle dei test	69
Indice analitico		71

1 Introduzione

1.1 Scopo del documento

Questo documento si rivolge a membri e amministratori dei team di sviluppo di progetti software, oltre che a stakeholders esterni. Il documento descrive *BugBoard26*, una piattaforma per la gestione delle issue all'interno di progetti software. La piattaforma consente ad utenti autenticati di segnalare problemi in un progetto, monitorarne lo stato, assegnarli a membri del team e tenere traccia delle attività di risoluzione. I membri del team di sviluppo sono i soli a dover utilizzare il software.

1.2 Campo di applicazione del sistema

Il sistema software in fase di sviluppo sarà indicato come System Under Development (SUD) o semplicemente come *BugBoard26*. *BugBoard26* dovrà svolgere le seguenti funzionalità :

1. Provvederà all'autenticazione dell'utente, consentendogli di accedere solo a quelle funzionalità che gli competono.
2. Gestirà un sistema di registrazione accessibile ai soli amministratori.
3. Tutti gli utenti autenticati potranno segnalare una issue.
4. La piattaforma sarà provvista di una vista riepilogativa delle issue, con possibilità di filtrare, ordinare e ricercare.
5. Il sistema permetterà ad utenti amministratori di assegnare issue ad un membro del team.
6. Durante la fase di assegnazione di una issue, il sistema suggerirà l'utenza con il carico di lavoro più basso.
7. Il sistema potrà inviare notifiche agli utenti.
8. Il sistema genererà report mensili sull'attività del team.
9. Il sistema permetterà agli amministratori di visualizzare i report sul team.

BugBoard26 non si occupa dello sviluppo o della risoluzione delle issue, ma esclusivamente della loro gestione, assegnazione e monitoraggio.

1.3 Definizioni, sinonimi adottati e abbreviazioni

1.3.1 Definizioni (Glossario)

- **Amministratore:** utente con privilegi elevati che si occupa della gestione globale del team, dell'assegnazione delle issue e della visualizzazione dei report di performance mensili. Può eseguire qualsiasi operazione standard dei membri del team, oltre alla creazione esclusiva di nuove utenze all'interno del sistema.
- **Attore:** rappresenta un'entità esterna (umana o artificiale) che interagisce direttamente con il SUD per conseguire uno specifico obiettivo di business all'interno di un caso d'uso.
- **Bug:** malfunzionamento tecnico del prodotto software; tipologia di issue indicante un difetto o un comportamento anomalo del codice sorgente che compromette il funzionamento programmato della piattaforma.
- **Issue:** problema o attività relativa al software, viene segnalata da un utente, contiene obbligatoriamente un titolo e una descrizione e appartiene a una delle seguenti quattro categorie:
 - **Question:** per richieste di chiarimenti tecnici o funzionali.
 - **Bug:** per segnalare un malfunzionamento del sistema.
 - **Documentation:** problemi o modifiche relative alla documentazione tecnica o utenze.
 - **Feature:** richiesta o suggerimento di nuove funzionalità o miglioramenti.
- **Metrica:** unità di misura quantitativa utilizzata per valutare l'efficienza complessiva del team di sviluppo. Le metriche integrate sono: Numero di issue segnalate, numero di issue gestite e tempo di risoluzione medio complessivo.
- **Mock-up:** rappresentazione visiva statica a bassa o media fedeltà di un'interfaccia grafica o di una schermata del sistema per illustrare la disposizione di campi, pulsanti e menu.
- **Notifica:** messaggio automatico generato asincronamente con cui la piattaforma comunica ad utenti autenticati alcune informazioni o eventi di interesse (es. l'avvenuta assegnazione di una issue).
- **Priorità:** parametro facoltativo di una issue (*High, Medium, Low*). Se inserito, definisce l'ordine di importanza e l'urgenza con cui il team di sviluppo deve prendere in carico e risolvere il problema.
- **Report:** visualizzazione o documento di riepilogo generato mensilmente che sintetizza dati aggregati e statistiche sulle attività svolte e sulla produttività del team, riservato agli amministratori.

- **Scenario:** sequenza di eventi che descrive l'interazione tra attori e sistema in un caso d'uso specifico.
- **Segretezza:** garantisce che le informazioni, i dati sensibili e le risorse di un sistema siano accessibili esclusivamente agli utenti che dispongono dell'autorizzazione esplicita.
- **Semplice:** intuitivo abbastanza da permettere di interagire con il software senza la necessità di supporto o documentazione aggiuntiva.
- **Sicuro:** soddisfa requisiti atti a proteggere l'accesso non autorizzato alle risorse e a garantire l'integrità dei dati.
- **Stakeholders:** persona o entità che ha un interesse nel prodotto software.
- **Stato della Issue:** la condizione in un dato momento della issue che ne indica la fase del ciclo di vita. Lo stato può assumere i seguenti tre valori sequenziali:
 - **Todo:** inteso come stato iniziale di una issue, inserito automaticamente al momento della creazione della segnalazione.
 - **In Progress:** inteso come stato intermedio nella risoluzione di una issue. Lo stato viene impostato quando un amministratore assegna la issue a un utente che dovrà lavorarci.
 - **Done:** inteso come stato finale che certifica la corretta e avvenuta risoluzione dell'issue.
- **System Under Development (SUD):** riferimento all'intero ecosistema applicativo software (frontend JavaFX, backend Spring Boot e database PostgreSQL) che si sta progettando e descrivendo nel documento SRS.
- **Utente autenticato:** persona membro del team di sviluppo regolarmente registrata ed autenticata nel sistema tramite procedura di Login.
- **Utente generico:** persona del team di sviluppo che non ha ancora effettuato l'accesso al sistema.
- **Vista:** l'insieme di informazioni e schermate dell'interfaccia utente accessibili per ciascun utente in base al proprio ruolo.
- **Workload:** indicatore quantitativo calcolato sul numero di issue in stato *In Progress* assegnate a ogni singolo utente, utilizzato dal sistema per suggerire l'operatore più idoneo a ricevere una nuova assegnazione.

1.3.2 Sinonimi adottati

- Membro del team, utente, utente generico, utente registrato.
- Lista, elenco, vista delle issue, riepilogo issue.
- Ospite, visitatore, stakeholder esterno, stakeholder generico.
- Sistema software, applicativo software, prodotto software, SUD, piattaforma, BugBoard26.
- Autenticazione, log in, accesso.
- Amministratore, admin, utente con permessi aggiuntivi.

1.3.3 Abbreviazioni

- **Admin**: abbreviazione di utente amministratore.
- **SUD**: System Under Development.
- **SRS**: Software Requirements Specifications.
- **DBMS**: Database Management System.
- **UML**: Unified Modeling Language.
- **MU**: Mock-up.

1.4 Riferimenti

Il presente documento è stato redatto in conformità allo standard IEEE Std 830-1998. Per la definizione dettagliata dei casi d'uso si è fatto riferimento alle tabelle di Alistair Cockburn.

Riferimenti bibliografici per la stesura del documento:

- *IEEE Std 830-1998 (Revision of IEEE Std 830-1993) – IEEE Recommended Practice for Software Requirements Specifications.*
- *Cockburn, A. (2001). Writing Effective Use Cases. Addison-Wesley.*

1.5 Panoramica del documento

La restante parte di questo documento è suddivisa in due ulteriori capitoli che descrivono le caratteristiche funzionali di *BugBoard26*.

"2. Descrizione generale" è composto dai seguenti paragrafi.

Prospettiva del prodotto	Illustra l'ambito in cui <i>BugBoard26</i> si va ad inquadrare e sarà utilizzato
Funzionalità del prodotto	Esplicita le funzioni implementate da <i>BugBoard26</i>
Utenti e attori del sistema	Individuazione e caratterizzazione di tutti gli utenti (attori) del sistema
Vincoli generali e assunzioni	Indica ciò a cui deve essere vincolato il prodotto software

In "**3. Requisiti specifici**" saranno dettagliate tutte le funzionalità che *BugBoard26* realizza. Il capitolo è composto dai seguenti paragrafi.

Requisiti funzionali	Si scende ad un livello di maggior dettaglio nella descrizione delle funzioni del prodotto software.
Requisiti non funzionali	Esplicita i requisiti non funzionali del prodotto software <i>BugBoard26</i>
Modellazione dei casi d'uso	Si modellano, tramite UseCase Diagram, tutti i casi d'uso del prodotto software.
Formalizzazione di casi d'uso significativi	Si scende ad un livello di maggiore dettaglio nella descrizione dei casi d'uso utilizzando le tabelle di Cockburn.
Prototipazione visuale	Si descriveranno visualmente le risposte del sistema nel suo utilizzo attraversando il flusso dei casi d'uso.

In "**4. Documento di Design del Sistema**" vengono illustrate l'architettura generale del sistema e le principali scelte tecnologiche adottate.

Analisi dell'architettura	Descrive la struttura architeturale del sistema, evidenziando i componenti Client, Server e Persistenza e le loro interazioni.
Scelte tecnologiche e motivazioni	Presenta le tecnologie adottate e le motivazioni che hanno guidato le decisioni progettuali.

In "**5. Documento di Design del Software**" vengono descritti in dettaglio gli aspetti di progettazione software e di implementazione del sistema.

Schema di persistenza dei dati	Descrive la struttura del database e le relazioni tra le entità del sistema.
Class Diagram di Design	Presenta i diagrammi UML delle classi coinvolte nelle principali funzionalità del sistema.

Package Diagram Frontend	Mostra l'organizzazione dei package del client e le relative dipendenze.
Scelte di software design	Illustra i design pattern e i principi architetturali adottati durante lo sviluppo.
Strumento di Versioning	Descrive l'organizzazione del repository GitHub e il processo di versionamento.
Containerizzazione, build automatica e Cloud Deploy	Presenta la strategia di build, containerizzazione e distribuzione del sistema su infrastruttura cloud.
Report di qualità del codice	Riporta le metriche e i risultati dell'analisi della qualità del codice tramite SonarQube.

2 Descrizione generale

2.1 Prospettiva del prodotto

Il sistema *BugBoard26* è un prodotto software autonomo progettato per la gestione interna delle issue nei team di sviluppo software. Non fa parte di un sistema più grande e funziona indipendentemente.

Tutti gli utenti devono autenticarsi prima di accedere alle funzionalità.

2.2 Funzionalità del prodotto

BugBoard26 deve fornire le funzionalità di seguito riportate, che per motivi di chiarezza sono state organizzate in categorie:

- Categoria Autenticazione - *BugBoard26* consente l'autenticazione dell'utente, consentendogli di accedere solo alle funzionalità che gli competono.
 - **Autenticazione:** *BugBoard26* consente l'autenticazione dell'utente.
 - **Registrazione nuova utenza (Amministratore):** Un amministratore registra una nuova utenza.
- Categoria issue - *BugBoard26* gestisce la segnalazione e l'assegnazione di issue.
 - **Segnala issue:** Un utente autenticato può segnalare una issue, della quale deve inserire un titolo e una descrizione. Può anche impostare una priorità e un tipo.
 - **Assegna issue:** Un amministratore può assegnare una issue ad un membro del team.
 - **Suggerimento utente:** Il sistema mostra una lista di utenti ordinati per carico di lavoro crescente. Il primo della lista viene suggerito esplicitamente all'amministratore.
 - **Modifica stato issue:** Quando una issue viene assegnata ad un utente il sistema ne modifica lo stato.
- Categoria visualizzazioni - *BugBoard26* permette di visualizzare informazioni su issue e membri del team.
 - **Visualizza issue:** gli utenti autenticati possono visualizzare la lista delle issue.
 - **Visualizza report team:** Gli amministratori hanno accesso ai report mensili del team.
 - **Visualizza issue utente:** Gli utenti autenticati possono visualizzare le issue assegnategli da un amministratore.
 - **Visualizza notifiche:** Il sistema fornisce una dashboard per la visualizzazione e gestione delle proprie notifiche.

- Categoria Notifiche - *BugBoard26* gestisce l'invio di notifiche agli utenti.
 - Invio notifiche: Il sistema notifica un utente quando un amministratore gli assegna una nuova issue.
 - Lettura notifiche: Un utente può segnare le proprie notifiche come lette o come non lette.

2.3 Utenti e attori del sistema

La seguente tabella definisce gli utenti principali del sistema, descrivendone il possibile utilizzo e la competenza tecnica richiesta. L'insieme risulta esaustivo, non è previsto, quindi, che utenti non specificati in questa sezione utilizzino il sistema. Per "competenze basilari di utilizzo di applicativi software" si intende una media familiarità con strumenti informatici di base. Non è prevista una curva di apprendimento sull'utilizzo di *BugBoard26*.

Membro del team autenticato		
Descrizione	Esperienza e competenze tecniche	Principali interazioni con il sistema
È un membro del team di sviluppo software. Può autenticarsi per segnalare nuove issue e visualizzarle	Conoscenza base del dominio del software. Competenze basilari di utilizzo di applicativi software.	Segnala issue; visualizzare riepilogo issue; visualizzare le issue che gli sono state assegnate; ricevere notifiche dal sistema
Amministratore		
Descrizione	Esperienza e competenze tecniche	Principali interazioni con il sistema
È un utente autenticato con privilegi estesi. Oltre alle funzionalità dell'utente, può creare nuove utenze, assegnare issue ai membri del team e visualizzare report mensili sui membri del team.	Conoscenza base del dominio del software. Competenze basilari di utilizzo di applicativi software.	Tutte le interazioni di un membro del team; assegna issue; visualizza report mensili
Membro del team non autenticato		
Descrizione	Esperienza e competenze tecniche	Principali interazioni con il sistema
È un utente che non ha effettuato l'accesso al sistema	Competenze basilari di utilizzo di applicativi software.	Effettuare il log in.

3 Requisiti specifici

3.1 Requisiti funzionali

3.1.1 RE01 - Log in utente

Utenti per funzionalità		
Utente	Utente autenticato	Amministratore
✓		
Funzione		Gestione dell'autenticazione utente
Descrizione		Il sistema deve permettere agli utenti di accedere al sistema tramite email e password. Solo utenti con credenziali valide possono accedere al sistema
Input		Email e password
Fonte		Interfaccia di autenticazione
Output		Accesso al sistema con vista personalizzata o messaggio di errore
Azione		Verifica la validità delle credenziali permettendo l'accesso all'utente
Precondizione		Utenza registrata
Effetti collaterali		Nessuno
Use Case		Vedi Use Case relativo a RE01

3.1.2 RE02 - Nuova utenza (Amministratore)

Utenti per funzionalità		
Utente	Utente autenticato	Amministratore
		✓

Funzione	Registrazione nuova utenza
Descrizione	Il sistema deve permettere di registrare nuove utenze all'interno del sistema software.
Input	Email e ruolo
Fonte	Interfaccia con vista dedicata agli utenti amministratori
Output	Visualizza messaggio di avvenuta registrazione della nuova utenza o visualizza messaggio di errore
Azione	Attivazione della nuova utenza con possibilità di autenticazione
Richiede	Credenziali conformi alle regole imposte dal sistema software
Precondizione	L'utente deve essere autenticato con permessi di amministratore
Effetti collaterali	Il sistema inserisce un nuovo utente con credenziali riservate nel sistema
Use Case	Vedi Use Case relativo a RE02

3.1.3 RE03 - Segnala issue

Utenti per funzionalità		
Utente	Utente autenticato	Amministratore
	✓	✓

Funzione	Segnalazione di una nuova issue
Descrizione	Il sistema deve permettere ad un utente di segnalare una nuova issue, specificando titolo, descrizione e, eventualmente, impostare un tipo e una priorità. Tipo: • Bug • Documentation • Feature • Question . Priorità: • High • Medium • Low .
Input	Titolo, descrizione e opzionalmente tipo e priorità
Fonte	Interfaccia utente di segnalazione issue
Output	Visualizza un messaggio di avventura segnalazione della nuova issue o un messaggio di errore
Azione	Il sistema crea la nuova issue
Richiede	Titolo e descrizione
Precondizione	L'utente deve essere autenticato
Effetti collaterali	Il sistema impostato lo stato della nuova issue a <i>Todo</i> .
Use Case	Vedi Use Case relativo a RE03
Descrizione dettagliata Cockburn	Vedi la descrizione dettagliata con le tabelle di Cockburn

3.1.4 RE04 - Assegna issue

Utenti per funzionalità		
Utente	Utente autenticato	Amministratore
		✓

Funzione	Assegnazione di una issue ad un membro del team registrato nel sistema
Descrizione	Il sistema deve permettere di assegnare issue a membri del team e modificare lo stato della issue da Todo a In progress. Stato: • Todo • In progress • Done
Input	Identificativo issue e Identificativo utente a cui è assegnata la issue
Fonte	Interfaccia utente di assegnazione issue
Output	Visualizza messaggio di avvertenza assegnazione della issue o messaggio di errore
Azione	Il sistema deve assegnare la issue all'utente segnalato
Precondizione	L'utente deve essere autenticato con permessi di amministratore.
Effetti collaterali	Il sistema invia una notifica all'utente cui è stata assegnata la issue e modifica lo stato della issue da <i>Todo</i> a <i>In Progress</i> .
Use Case	Vedi Use Case relativo a RE04

3.1.5 RE05 - Visualizza riepilogo issue

Utenti per funzionalità		
Utente	Utente autenticato	Amministratore
	✓	✓

Funzione	Visualizzazione del riepilogo delle issue
Descrizione	Il sistema deve permettere la visualizzazione di tutte le issue
Input	Richiesta di visualizzazione dell'elenco delle issue
Fonte	Interfaccia utente
Output	Elenco di tutte le issue presenti nel sistema
Azione	Il sistema deve gestire la visualizzazione delle issue presenti nel sistema
Precondizione	L'utente deve essere autenticato
Effetti collaterali	Nessuno
Use Case	Vedi Use Case relativo a RE05

3.1.6 RE06 - Visualizza report team

Utenti per funzionalità		
Utente	Utente autenticato	Amministratore
		✓
Funzione	Visualizzazione del report mensile del team	
Descrizione	Il sistema deve permettere la visualizzazione di metriche aggregate e dettagliate per ogni singolo utente	
Input	Mese richiesto	
Fonte	Interfaccia utente amministratore	
Output	Elenco del report mensile	
Azione	Il sistema deve visualizzare il report	
Precondizione	L'utente deve essere autenticato con permessi di amministratore	
Effetti collaterali	Nessuno	
Use Case	Vedi Use Case relativo a RE06	

3.1.7 RE07 - Utente visualizza le proprie issue assegnate

Utenti per funzionalità		
Utente	Utente autenticato	Amministratore
	✓	✓
Funzione	Visualizzazione delle issue assegnate ad un utente	
Descrizione	Il sistema deve permettere la visualizzazione ad un utente delle issue che gli sono state assegnate	
Input	Richiesta di visualizzazione	
Fonte	Interfaccia utente	
Output	Elenco delle issue	
Azione	Il sistema deve permettere la visualizzazione l'elenco	
Precondizione	L'utente deve essere autenticato	
Effetti collaterali	Nessuno	
Use Case	Vedi Use Case relativo a RE07	

3.1.8 RE08 - Filtra issue

Utenti per funzionalità		
Utente	Utente autenticato	Amministratore
	✓	✓

Funzione	Filtraggio delle issue
Descrizione	Il sistema deve permettere all'utente che intende visualizzare le issue di applicare un filtro in base ai seguenti criteri: • Tipologia • Priorità • Stato
Input	Seleziona il criterio
Fonte	Interfaccia utente nella vista di visualizzazione delle issue
Output	Elenco delle issue filtrate
Azione	Il sistema deve applicare il filtro e aggiornare l'elenco delle issue
Precondizione	L'utente deve essere autenticato e deve esistere almeno una issue nel sistema
Effetti collaterali	Nessuno
Use Case	Incluso in Visualizza riepilogo issue - RE05

3.1.9 RE09 - Ordina issue

Utenti per funzionalità		
Utente	Utente autenticato	Amministratore
	✓	✓

Funzione	Ordinamento delle issue
Descrizione	Il sistema deve permettere all'utente di ordinare le issue visualizzate secondo i seguenti criteri: • Titolo • Descrizione • Tipologia • Priorità • Stato • Data di creazione • Data di risoluzione
Input	Seleziona il criterio di ordinamento
Fonte	Interfaccia utente nella vista di visualizzazione delle issue
Output	Elenco delle issue ordinate
Azione	Il sistema deve mostrare l'elenco delle issue ordinate
Precondizione	L'utente deve essere autenticato e deve esistere almeno una issue nel sistema
Effetti collaterali	Nessuno
Use Case	Incluso in Visualizza riepilogo issue - RE05

3.1.10 RE10 - Ricerca issue

Utenti per funzionalità		
Utente	Utente autenticato	Amministratore
	✓	✓

Funzione	Ricerca delle issue
Descrizione	Il sistema deve permettere all'utente di effettuare una ricerca delle issue secondo i seguenti criteri: • Titolo • Descrizione
Input	Chiave di ricerca
Fonte	Interfaccia utente nella vista di visualizzazione delle issue
Output	Elenco delle issue trovate
Azione	Il sistema deve mostrare l'elenco delle issue ritenute pertinenti con la ricerca
Precondizione	L'utente deve essere autenticato.
Effetti collaterali	Nessuno
Use Case	Incluso in Visualizza riepilogo issue - RE05

3.1.11 RE11 - Visualizza notifiche

Utenti per funzionalità		
Utente	Utente autenticato	Amministratore
	✓	✓

Funzione	Visualizzazione elenco notifiche
Descrizione	Il sistema permette la visualizzazione dell'elenco di tutte le notifiche inviate all'utente autenticato. Con visualizzazione specifica tra notifiche lette e non lette.
Input	Seleziona pagina di notifiche
Fonte	Interfaccia utente nella vista di visualizzazione delle notifiche
Output	Elenco delle notifiche
Azione	Il sistema deve mostrare l'elenco delle notifiche
Precondizione	L'utente deve essere autenticato.
Effetti collaterali	Nessuno
Use Case	Incluso in Visualizza elenco notifiche - RE11

3.1.12 RE12 - Lettura notifiche

Utenti per funzionalità		
Utente	Utente autenticato	Amministratore
	✓	✓

Funzione	Una notifica può essere impostata come letta
Descrizione	Il sistema permette all'utente di impostare una notifica come letta o come non letta in qualsiasi momento.
Input	Seleziona tasto "segna come letto" o "segna come non letto"
Fonte	Interfaccia utente nella vista di visualizzazione delle notifiche
Output	Parametro di lettura modificato
Azione	Il sistema deve mostrare l'elenco aggiornato delle notifiche con i realtivi parametri di lettura
Precondizione	L'utente deve essere autenticato.
Effetti collaterali	Nessuno
Use Case	

3.1.13 RE13 - Workload

Utenti per funzionalità		
Utente	Utente autenticato	Amministratore
		✓

Funzione	Il sistema suggerisce l'utente con il carico di lavoro più basso
Descrizione	Il sistema permette all'amministratore di scegliere l'utente a cui assegnare la issue in una lista di utenti ordinata per carico di lavoro crescente.
Input	Assegna issue
Fonte	Interfaccia utente nella assegnazione di una issue
Output	Lista ordinata di utenti per carico di lavoro
Azione	Il sistema deve mostrare l'elenco degli utenti ordinata in base al carico di lavoro in ordine crescente
Precondizione	L'utente deve essere autenticato come amministratore.
Effetti collaterali	Nessuno
Use Case	

3.2 Modellazione dei casi d'uso tramite Use Case Diagram

3.2.1 Use Case Diagram

Di seguito è riportato il diagramma dei casi d'uso del System Under Development. Le prossime sezioni del documento (3.3.1 - 3.3.5) presenteranno alcuni Use Case Diagram parziali, estratti dal diagramma complessivo, con l'unico scopo di facilitare la lettura e la comprensione dei singoli casi d'uso.

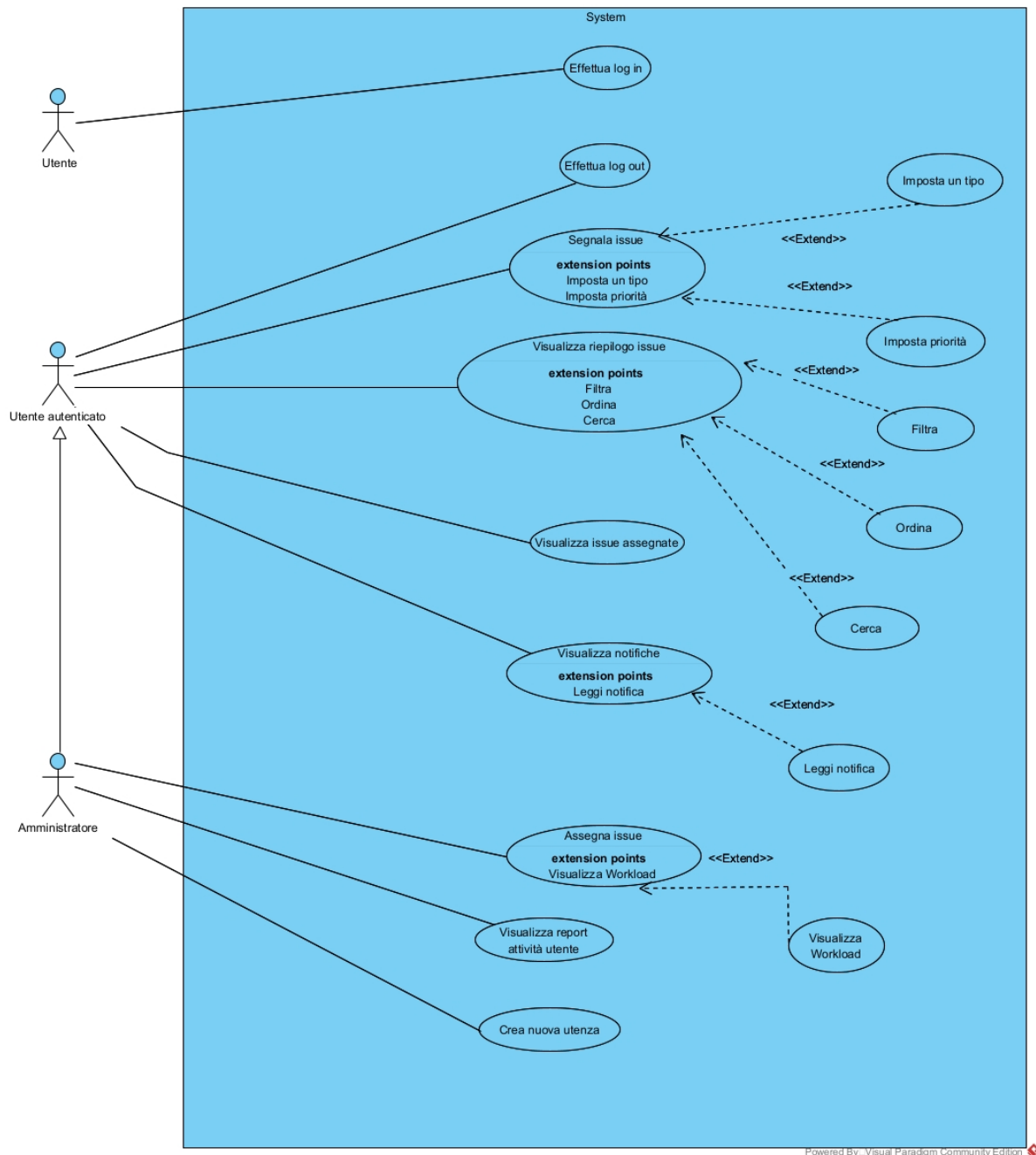


Figura 1: Use Case Diagram

3.2.2 Use Case Diagram parziale 1

Il seguente Use Case è un estratto dello Use Case generale. Non aggiunge informazioni al caso d'uso specifico. Serve solamente a focalizzare l'attenzione sul caso/i d'uso specifico/i. I casi d'uso mostrati sono due: *Log in* e *Registra nuova utenza*.

- *Log in*. L'attore principale è un Utente, esso può effettuare un Log in per autenticarsi nel sistema ed avere accesso alle funzionalità garantite dai privilegi della sua utenza. Per

effettuare il Log in necessita di una email e una password corrispondenti ad un utente registrato e attivo nel sistema.

- *Registra nuova utenza.* L'attore principale è un amministratore. Esso può registrare nuove utenze nel sistema specificando una email il ruolo.

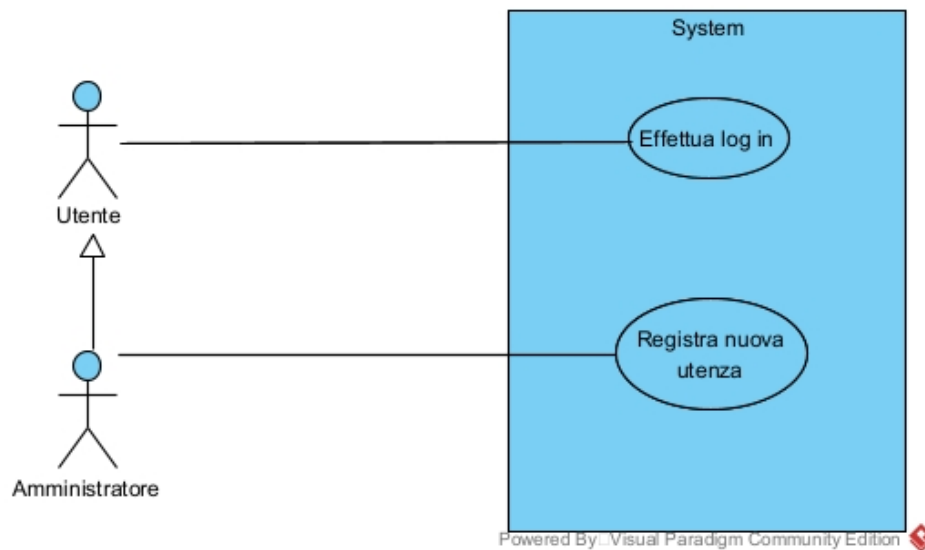


Figura 2: Use Case1 - [Requisito funzionale 3.1.1 - Log in utente](#) e [Requisito funzionale 3.1.2 - Crea nuova utenza](#)

3.2.3 Use Case Diagram parziale 2

Il seguente Use Case è un estratto dello Use Case generale. Non aggiunge informazioni al caso d'uso specifico. Serve solamente a focalizzare l'attenzione sul caso/i d'uso specifico/i. In particolare, si pone l'attenzione sulla segnalazione di una issue. L'attore principale è un utente autenticato. Esso può segnalare una issue, specificando un titolo e una descrizione. Può anche impostare una priorità e un tipo. La issue verrà inserita nel sistema e sarà visibile nell'elenco delle issue.

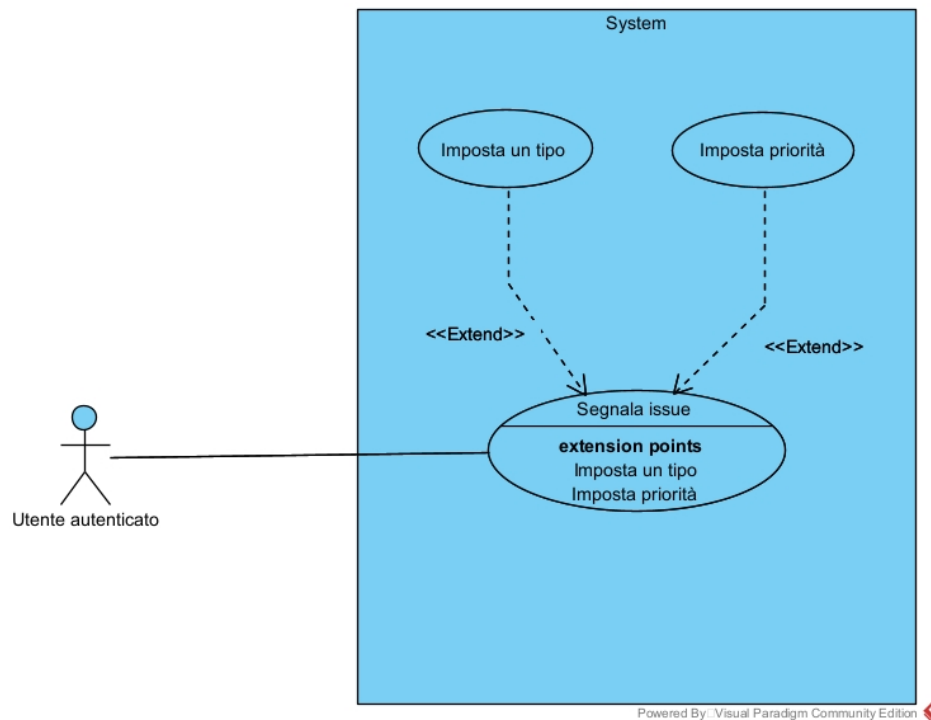


Figura 3: Use Case2 - [Requisito funzionale 3.1.3 - Segnala issue](#)

3.2.4 Use Case Diagram parziale 3

Il seguente Use Case è un estratto dello Use Case generale. Non aggiunge informazioni al caso d'uso specifico. Serve solamente a focalizzare l'attenzione sul caso/i d'uso specifico/i. I casi d'uso mostrati sono due: *Segnala issue* e *Visualizza issue assegnate*.

- *Assegna issue*. L'attore principale è un amministratore. Esso può assegnare una issue a membri del team, ammesso che la issue non risulti già assegnata. Il sistema modificherà lo stato della issue da *Todo* a *In Progress* e invierà una notifica all'utente al quale è stata assegnata la issue.
- *Visualizza issue assegnate*. L'attore principale è un utente autenticato. Esso può visualizzare l'elenco delle issue che gli sono state assegnate.

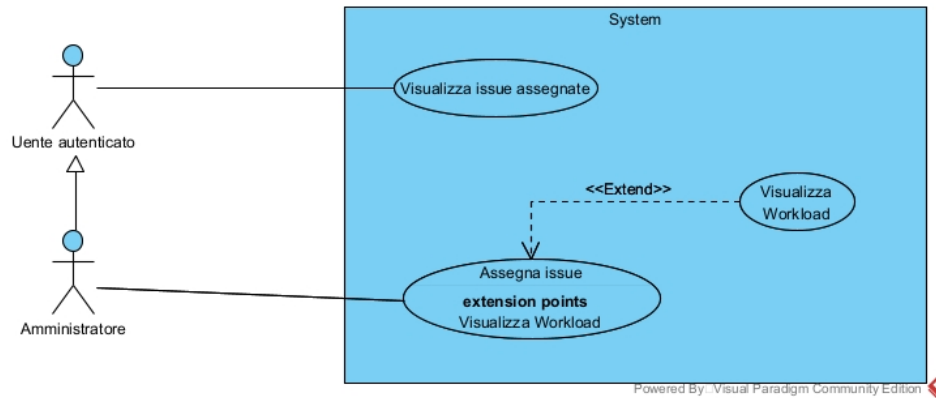


Figura 4: Use Case3 - [Requisito funzionale 3.1.4 - Assegna issue](#) e [Requisito funzionale 3.1.7 - Visualizza issue assegnate](#)

3.2.5 Use Case Diagram parziale 4

Il seguente Use Case è un estratto dello Use Case generale. Non aggiunge informazioni al caso d'uso specifico. Serve solamente a focalizzare l'attenzione sul caso/i d'uso specifico/i. I casi d'uso mostrati sono due: *Visualizza riepilogo issue* e *Visualizza report attività team*.

- *Visualizza riepilogo*. L'attore principale è un utente autenticato. Esso può visualizzare un elenco di tutte le issue e relativi commenti, presenti nel sistema. L'elenco è dinamico, nel senso che è possibile ottenere una vista personalizzata delle issue, filtrando, ordinando o ricercando per parametri e parole chiave.
- *Visualizza report attività team*. L'attore principale è un amministratore. Esso può visualizzare i report mensili del team, con le metriche e i dettagli per singolo utente.

Il seguente caso d'uso integra i requisiti funzionali [RE08](#), [RE09](#) e [RE10](#), che non sono rappresentate come casi d'uso autonomi.

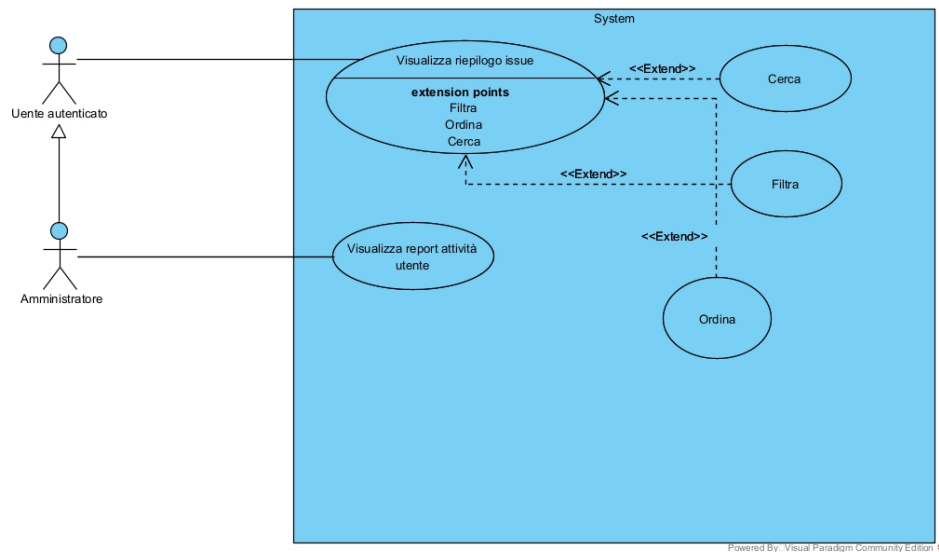


Figura 5: Use Case4 - [Requisito funzionale 3.1.5 - Visualizza riepilogo issue](#) e [Requisito funzionale 3.1.6 - Visualizza report team](#)

3.2.6 Use Case Diagram parziale 6

Il seguente Use Case è un estratto dello Use Case generale. Non aggiunge informazioni al caso d'uso specifico. Serve solamente a focalizzare l'attenzione sul caso/i d'uso specifico/i. In particolare, si pone l'attenzione sulla visualizzazione delle proprie notifiche.

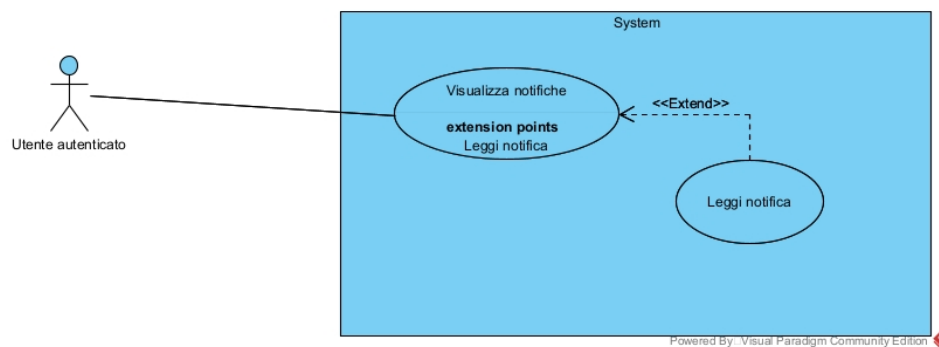


Figura 6: Use Case6 - [Requisito funzionale 3.1.11 - Visualizza Notifiche](#)

3.3 Requisiti non funzionali

Di seguito sono elencati i requisiti non funzionali del sistema *BugBoard26*, classificati secondo le categorie previste dallo standard IEEE Std 830-1998.

3.3.1 RNF01 - Sicurezza

Il sistema deve garantire la *segretezza* e l'*integrità* delle informazioni gestite, in quanto critiche per l'azienda. In particolare:

- L'accesso alle funzionalità del sistema deve essere consentito esclusivamente ad utenti autenticati, secondo i permessi associati al proprio ruolo (Utente o Amministratore).
- La gestione delle credenziali deve avvenire tramite un sistema che impedisca la memorizzazione delle password all'interno dell'applicazione.
- La creazione di nuove utenze non può avvenire in autonomia da parte degli utenti, ma richiede necessariamente l'intervento di un Amministratore.

3.3.2 RNF02 - Affidabilità

Il sistema deve garantire la corretta e costante disponibilità delle proprie funzionalità e ripristino automatico in caso di malfunzionamenti.

3.3.3 RNF03 - Prestazioni

Il sistema deve garantire tempi di risposta rapidi durante l'interazione con l'utente, in particolare nelle operazioni più frequenti quali la visualizzazione, il filtraggio, l'ordinamento e la ricerca delle issue. La gestione asincrona di alcune operazioni (es. l'invio delle notifiche a seguito dell'assegnazione di una issue), contribuisce a non penalizzare i tempi di risposta percepiti dall'utente durante le richieste principali.

3.3.4 RNF04 - Usabilità

L'interfaccia utente deve essere semplice ed intuitiva, tale da consentire l'utilizzo del sistema senza necessità di supporto o di una curva di apprendimento (vedi [Sezione 2.3 - Utenti e attori del sistema](#)).

3.3.5 RNF05 - Scalabilità

Il sistema deve essere in grado di supportare un aumento del numero di utenti, issue e notifiche gestite senza richiedere modifiche sostanziali alle funzionalità offerte. Inoltre, deve essere possibile aumentare le risorse computazionali dedicate ai componenti del sistema al fine di sostenere carichi di lavoro maggiori.

3.3.6 RNF06 - Portabilità e indipendenza tra i componenti

Il front-end e il back-end devono essere componenti indipendenti, comunicanti esclusivamente tramite API REST esposte dal back-end. Deve essere possibile sostituire o modificare il front-end senza alcun impatto sul back-end, e viceversa.

3.3.7 RNF07 - Manutenibilità

Il codice del sistema deve essere organizzato secondo un'architettura a livelli e principi di progettazione ad oggetti, al fine di favorire la leggibilità, l'estensibilità e il riutilizzo del codice.

3.4 Formalizzazione di casi d'uso significativi

Descrizione tramite tabelle di Cockburn dei casi d'uso significativi.

3.4.1 Tabella di Cockburn 1

Tabella di Cockburn del caso d'uso [Requisito funzionale 3.1.2 - Segnala issue](#)

Use Case 2	Segnala Issue		
Goal in Context	Permette agli utenti autenticati di creare una issue con titolo e descrizione e, facoltativamente, di inserire un tipo e assegnare un livello di priorità		
Preconditions	L'utente ha effettuato l'autenticazione nel sistema		
Success End Condition	L'issue viene registrata nel sistema, risultando visibile con lo stato iniziale di <i>Todo</i>		
Failed End Condition	L'issue non viene registrata nel sistema e viene visualizzato il messaggio di Errore: <i>Issue non creata</i>		
Primary Actor	Utente autenticato		
Trigger	L'utente autenticato sceglie di creare una nuova issue		
Main Scenario	Step	Utente autenticato	Risposta del sistema
	1	Accede alla sezione <i>Crea nuova issue</i>	
	2		Mostra MU_1
	3	Inserisce i campi obbligatori: titolo e descrizione	
	4	Clicca <i>Crea</i>	
	5		Crea la issue e Mostra MU_2
	6	Clicca <i>Ok</i>	
Extension #1 - Imposta una priorità (campo opzionale)	Step	Utente autenticato	Risposta del sistema
	3.a	Sceglie di selezionare una priorità	
	3.a.1		Torna allo step 4 del <i>Main Scenario</i>

Extension #2 - Allega un'immagine (campo opzionale)	Step	Utente autenticato	Risposta del sistema
	3.b	Sceglie di selezionare un tipo	
	3.b.1	Clicca <i>Crea</i>	
	3.b.2		Torna allo step 4 del <i>Main Scenario</i>
Extension #3 - Gestione Errore connessione con il Server	Step	Utente autenticato	Risposta del sistema
	4.c	Clicca su <i>Crea</i>	
	4.c.1		Mostra MU_3 e torna allo step 3 del <i>Main Scenario</i>
Extension #4 - Campo obbligatorio non specificato	Step	Utente autenticato	Risposta del sistema
	4.d	Clicca su <i>Crea</i>	
	4.d.1		Mostra MU_4 e torna allo step 3 del <i>Main Scenario</i>
Extension #5 - L'utente non conferma la creazione	Step	Utente autenticato	Risposta del sistema
	6.a	Clicca <i>Annulla</i>	
	6.a.1		Torna allo step 3 del <i>Main Scenario</i>

3.4.2 Tabella di Cockburn 2

Tabella di Cockburn del caso d'uso [Requisito funzionale 3.1.4 - Assegna Issue](#)

Use Case 3	Assegna Issue
Goal in Context	Permette all'amministratore di assegnare una issue ad un utente
Preconditions	L'amministratore ha effettuato l'accesso al sistema
Success End Condition	L'issue viene assegnata all'utente prescelto e il suo stato passa a <i>In Progress</i>
Failed End Condition	L'assegnazione non viene completata e viene visualizzato il messaggio di errore: <i>Impossibile completare l'assegnazione</i>
Primary Actor	Amministratore
Trigger	L'amministratore clicca su <i>Assegna Issue</i>

Main Scenario	Step	Amministratore	Risposta del sistema
	1	Accede alla sezione <i>Assegna Issue</i>	
	2		Mostra MU_1
	3	Seleziona la issue da assegnare e seleziona l'utente	
	4	Clicca <i>Assegna</i>	
	5		Mostra MU_2
	6	Clicca <i>OK</i>	
Extension #1 - Utente e/o Issue non selezionati	Step	Amministratore	Risposta del sistema
	4.a	Non seleziona l'utente e/o la issue	
	4.a.1	Clicca su <i>Assegna</i>	
	4.a.2		Mostra MU_3 e torna allo step 3 del <i>Main Scenario</i>
Extension #2 - Errore di rete	Step	Amministratore	Risposta del sistema
	4.b	Clicca su <i>Assegna</i>	
	4.b.1		Mostra MU_4
	4.b.2	Clicca <i>OK</i>	
	4.b.3		Torna allo step 3 del <i>Main Scenario</i>

3.5 Prototipazione visuale del caso d'uso 3.3.3 Segnala Issue

Le seguenti figure rappresentano la prototipazione visuale via Mock-up delle relative interfacce utente. Non sono da intendersi come rappresentativi dell'aspetto finale della schermata, ma solamente un mezzo per comunicare l'idea dell'interfaccia grafica del prodotto software.

3.5.1 Segnala Issue - Mockup 1



The mockup shows a form titled "Crea Issue" with the instruction "Inserisci i dati richiesti per la segnalazione". The form contains four input fields: "Titolo*" (text), "Descrizione*" (text), "Tipo" (dropdown menu with "Bug" selected), and "Priorità" (dropdown menu with "High" selected). A blue "INVIA" button is located at the bottom right.

Crea Issue

Inserisci i dati richiesti per la segnalazione

Titolo*

Titolo

Descrizione*

Titolo

Tipo

Bug ▼

Priorità

High ▼

INVIA

Figura 7: Mockup 1 - Crea issue riferito a [3.4.1 Tabella di Cockburn 1 - Step 2 del Main Scenario](#)

3.5.2 Segnala Issue - Mockup 2

The mockup shows a 'Create Issue' form with the following elements:

- Title***: A text input field containing the placeholder text 'Titolo'.
- Description***: A text area for the issue description.
- Priority**: A dropdown menu currently set to 'High'.
- Submit Button**: A button labeled 'INVIA' at the bottom right.
- Success Message Overlay**: A modal box with a blue information icon and the text 'La Issue è stata creata con successo!' (The issue was created successfully!). It includes an 'Ok' button.

Figura 8: Mockup 2 - Crea issue riferito a [3.4.1 Tabella di Cockburn 1 - Step 5](#)

3.5.3 Segnala Issue - Mockup 3

The mockup shows a 'Crea Issue' form with the following elements:

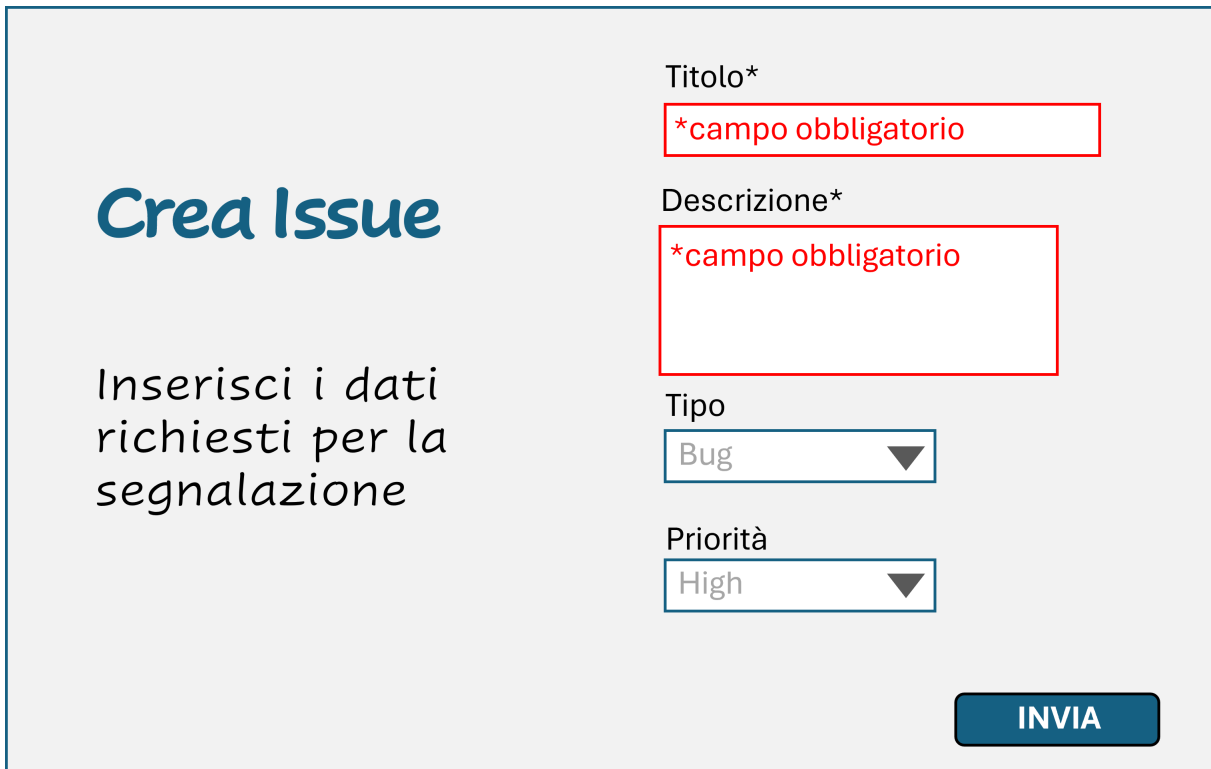
- Titolo***: A text input field containing the placeholder text 'Titolo'.
- Descrizione***: A text area for the issue description.
- Priorità**: A dropdown menu currently set to 'High'.
- IN VIA**: A button at the bottom right of the form.

An error message dialog is overlaid on the form:

- Icona**: A blue circle with a white lowercase 'i'.
- Testo**: **Impossibile salvare la issue. Il Server non risponde.** (Impossible to save the issue. The server does not respond.)
- Botone**: A blue button labeled **Ok**.

Figura 9: Mockup 3 - Crea issue riferito a [3.4.1 Tabella di Cockburn 1 - Step 4.c.1](#)

3.5.4 Segnala Issue - Mockup 4



The mockup shows a form titled "Crea Issue" with the instruction "Inserisci i dati richiesti per la segnalazione". The form contains four fields: "Titolo*" (required), "Descrizione*" (required), "Tipo", and "Priorità". The "Titolo*" and "Descrizione*" fields are highlighted with red borders and red text indicating they are mandatory. The "Tipo" field is a dropdown menu with "Bug" selected. The "Priorità" field is a dropdown menu with "High" selected. An "INVIA" button is located at the bottom right of the form.

Crea Issue

Inserisci i dati richiesti per la segnalazione

Titolo*
*campo obbligatorio

Descrizione*
*campo obbligatorio

Tipo
Bug ▼

Priorità
High ▼

INVIA

Figura 10: Mockup 4 - Crea issue riferito a [3.4.1 Tabella di Cockburn 1 - Step 4.b.1](#)

3.6 Prototipazione visuale del caso d'uso Assegna Issue

Le seguenti figure rappresentano la prototipazione visuale via Mock-up delle relative interfacce utente. Non sono da intendersi come rappresentativi dell'aspetto finale della schermata, ma solamente un mezzo per comunicare l'idea dell'interfaccia grafica del prodotto software.

3.6.1 Assegna Issue - Mockup 1

Assegna Issue

Seleziona una Issue ed un utente a cui assegnarla

Seleziona Issue

Issue1
Issue2
Issue3
Issue4

Seleziona Utente

Utente1
Utente2
Utente3
Utente4

INVIA

Figura 11: Mockup 1 - Assegna issue riferito a [Tabella di Cockburn 2 - Step 2 del Main Scenario](#)

3.6.2 Assegna Issue - Mockup 2

The mockup shows a web form titled "Assegna Issue". On the left, there is a vertical label "Seleziona Issue" and a list of "utenti" (users) with "assegnati" (assigned) next to them. The main form area contains two input fields: "Issue4" and "Utente4". A dark blue button labeled "IN VIA" is at the bottom right. A light gray modal box is overlaid on the form, containing a blue circle with a white 'i' icon, the text "Issue assegnata con successo!" in green, and a dark blue button labeled "Ok".

Figura 12: Mockup 2 - Assegna issue riferito a [Tabella di Cockburn 2 - Step 5 del Main Scenario](#)

3.6.3 Assegna Issue - Mockup 3

The mockup shows a web interface for assigning an issue. The main title is "Assegna Issue". Below it, there is a form with two input fields: "Issue4" and "Utente4". A modal error message is displayed in the center, stating: "Errore: Selezionare la issue da assegnare e l'utente a cui assegnarla." with an "Ok" button. To the right of the error message, there is a dropdown menu labeled "ente" with a downward arrow. At the bottom right, there is a button labeled "INVIA".

Seleziona
Issue
utente
assegnare

Errore: Selezionare la issue da assegnare e l'utente a cui assegnarla.

Ok

Issue4

Utente4

ente

INVIA

Figura 13: Mockup 3 - Assegna issue riferito a [Tabella di Cockburn 2 - Step 4.a.2](#)

3.6.4 Assegna Issue - Mockup 4

The mockup shows a web form titled "Assegna Issue". On the left, there is a vertical list of labels: "Seleziona", "Issue", "utente", and "assegnato". The form contains two input fields: "Issue4" and "Utente4". To the right of these fields is a dropdown menu labeled "Utente" with a downward arrow. Below the dropdown are three empty input fields. At the bottom right of the form is a button labeled "INVIA". A modal error message is displayed in the center, featuring a blue information icon and the text: "Errore di rete: Impossibile connettersi al Server. Issue non assegnata, riprova." with an "Ok" button.

Figura 14: Mockup 4 - Assegna issue riferito a [Tabella di Cockburn 2 - Step 4.b.1](#)

4 Documento di design del sistema

4.1 Analisi dell'Architettura

Il progetto BugBoard26 è stato sviluppato con un'architettura *client-server* (fig. 1) organizzata su tre livelli che comunicano tramite API REST. Tale organizzazione è suddivisa in:

- Presentation Layer (Client)
- Bussines Logic Layer (Server)
- Data Layer (Persistenza)

Questa scelta consente di ottenere numerosi vantaggi pratici. Tra i principali si evidenzia la centralizzazione dei dati: le informazioni vengono gestite dal server e distribuite ai client che ne fanno richiesta, garantendo così maggiore controllo e coerenza. Un ulteriore beneficio riguarda la sicurezza, in particolare per quanto concerne la gestione dell'autenticazione degli utenti, che è affidata al server e risulta quindi più efficace e affidabile. Un altro aspetto particolarmente rilevante è la scalabilità: il modello permette infatti di aggiungere nuovi client o server senza apportare modifiche sostanziali all'intero sistema. Per tutti questi motivi, la decisione di utilizzare il modello Client-Server rappresenta la soluzione più coerente per lo sviluppo del software.



Figura 15: Schema architetturale 1 - Architettura Client-Server

4.1.1 Server

Il sistema adotta un'architettura basata sul **pattern MVC** (Model-View-Controller), estesa attraverso l'introduzione di ulteriori livelli applicativi, come Service, DTO, Mapper e moduli di autenticazione. Questa organizzazione consente una separazione ancora più marcata delle responsabilità: i **Controller** si occupano della gestione delle richieste HTTP, i **Services** della logica di business, i **DTO** e **Mapper** facilitano lo scambio e la trasformazione dei dati, la componente **View** è rappresentata dal processo di serializzazione (da Object a JSON) e deserializzazione (da JSON a Object) dei dati trasferiti dal o verso il client, mentre la componente **Repositories** rappresenta l'interfaccia che fornisce i servizi per la comunicazione con il sistema di persistenza dei dati.

L'architettura appena illustrata è rappresentata dal seguente Component Diagram UML.

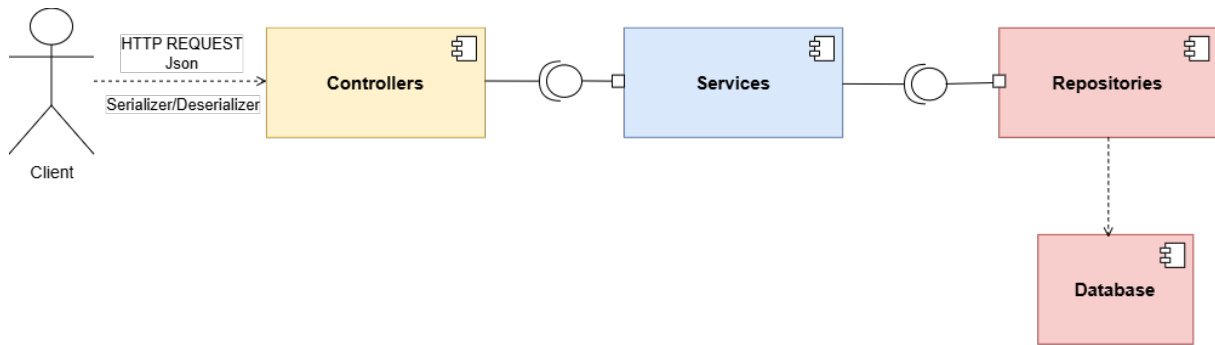


Figura 16: Schema architetturale 2 - Component Diagram del backend

4.1.2 Persistenza

La persistenza fisica dei dati è sviluppata mediante l'utilizzo di un DBMS relazionale disaccoppiato dal resto del sistema tramite l'interfaccia Repository. Il database relazionale garantisce la persistenza dei dati in modo sicuro. L'interazione con esso avviene tramite il livello **Repositories**, che incapsula le operazioni di accesso ai dati, mantenendo separata la logica applicativa dalla persistenza dei dati.

4.1.3 Client

L'architettura del *frontend* è basata sul pattern architetturale MVC (*Model-View-Controller*) esteso con moduli ausiliari per una corretta separazione delle responsabilità. In particolare, il modulo ViewController, insieme alle risorse FXML, costituiscono il livello di presentazione. I ViewController gestiscono le pagine FXML garantendo il corretto funzionamento della GUI. Il modulo Service si occupa della logica di business, riducendo le responsabilità dei Controller, che ricevono gli eventi generati dalla View, richiamano i servizi che comunicano con il back-end tramite le API REST esposte, e aggiornano il Model con i dati ricevuti. L'architettura appena illustrata è rappresentata dal seguente Component Diagram UML.

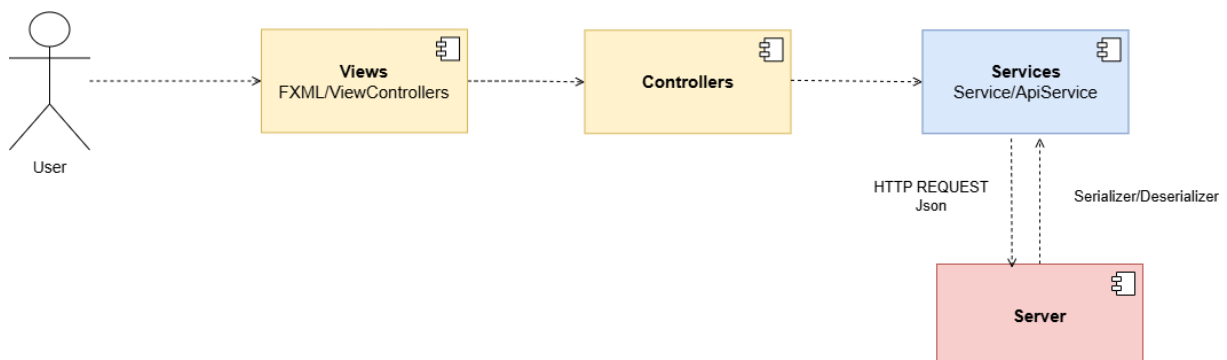


Figura 17: Schema architetturale 3 - Component Diagram del frontend

4.2 Scelte tecnologiche e motivazioni

La scelta tecnologica più rilevante, riferita all'intero sistema, riguarda la gestione dell'autenticazione. L'autenticazione è stata gestita interamente tramite **Amazon Cognito**, configurato su un server nella regione di Milano. Tramite un'apposita pagina UI del frontend, l'utente può richiedere di effettuare il login: la richiesta lo indirizza verso la *Hosting UI* di Cognito, una pagina di autenticazione esterna al sistema *BugBoard26*. Contestualmente, il frontend apre un server temporaneo in ascolto sulla porta *9090*, in attesa della risposta (positiva o negativa) da parte di Cognito.

Come indicato nel documento dei requisiti, il sistema non permette la registrazione spontanea a BugBoard26: la creazione di una nuova utenza richiede necessariamente l'intervento di un Amministratore. La registrazione di una nuova utenza (sia essa di ruolo Admin che User) avviene interamente all'interno di BugBoard26: il frontend acquisisce email e ruolo del nuovo utente, mentre il backend, oltre a persistere i dati nel database, invia una richiesta API a Cognito per aggiungere l'utente al pool di utenti già esistente. Cognito provvede quindi all'invio di un'e-mail al nuovo utente contenente una password temporanea, forzando il cambio della password al primo accesso.

Questa scelta garantisce un sistema di autenticazione semplice e, soprattutto, sicuro, poiché **nessuna password viene memorizzata all'interno del sistema**.

Nei paragrafi successivi vengono presentate le principali scelte tecnologiche adottate nel sistema, suddivise per *Client*, *Persistenza* e *Server*.

4.2.1 Server

Per lo sviluppo del **backend** abbiamo utilizzato Spring Boot (v. 3.2.2), un framework Java che facilita la creazione di API REST grazie al supporto per la gestione di richieste HTTP e l'integrazioni di componenti Spring come Spring MVC. Grazie all'integrazione di Spring Data JPA (Jakarta Persistence API) e Hibernate, i Repositories si occupano delle operazioni di lettura e scrittura dei dati (CRUD), delegando al sistema di persistenza la sola gestione fisica delle informazioni. Tale approccio migliora la manutenibilità, la scalabilità e la chiarezza del codice.

Il Server è stato pensato e predisposto per essere containerizzato (con Docker) e messo in opera utilizzando servizi di Cloud Computing (Amazon Web Services).

4.2.2 Persistenza

Per la persistenza dei dati è stato scelto PostgreSQL, un DBMS relazionale open-source, affidabile e altamente stabile. Questa scelta è coerente con la natura fortemente relazionale del dominio: le entità principali (Users, Issues, Notifications) sono legate da relazioni ben definite con vincoli di integrità referenziale (es. una Issue ha sempre un reporter e opionalmente

un assegnatario) PostgreSQL è stato scelto per diversi fattori. Inoltre, PostgreSQL si integra facilmente con il framework Spring Boot e tecnologie JPA/Hibernate.

4.2.3 Client

Per lo sviluppo del front-end è stato scelto JavaFX, un framework per la realizzazione di applicazioni desktop in Java. Tale scelta è stata motivata da diversi fattori. Tra i principali motivi, essendo il *backend* sviluppato in Java, l'utilizzo di JavaFX permette di avere **coerenza tecnologica** e di riutilizzare competenze e strumenti su entrambe le componenti del sistema. Altra motivazione decisiva è il supporto nativo di JavaFX con il pattern *MVC*, che favorisce naturalmente la separazione tra interfaccia grafica e logica di presentazione, tramite il formato FXML. Inoltre, rispetto ad alternative come Swing, JavaFX offre un supporto più moderno per componenti grafici, tramite CSS, e gestione di layout responsive, mantenendo la stessa capacità di comunicare con il back-end esclusivamente tramite chiamate HTTP verso le API REST esposte, senza alcuna dipendenza diretta dal codice del server.

5 Documento di Design del Software

5.1 Descrizione dello schema di persistenza dei dati

Nella Figura 15 viene mostrato il diagramma UML del database.

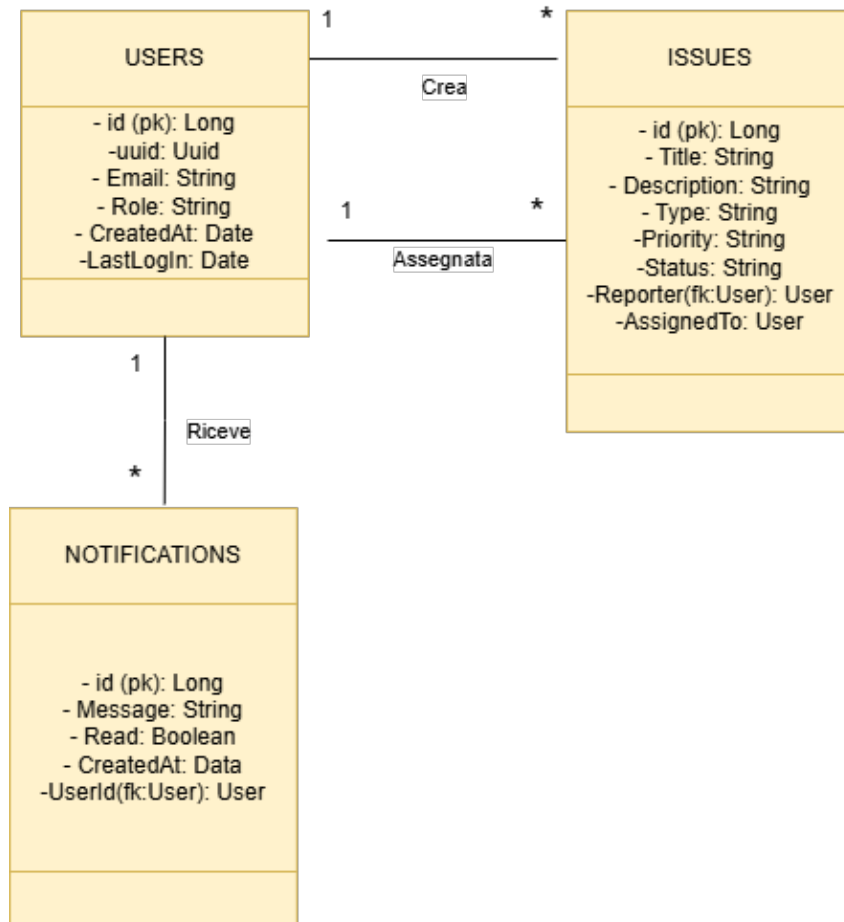


Figura 18: Schema architetturale 3 - Diagramma UML Database

5.2 Class Diagram di Design

Di seguito sono riportati i class diagram del Server. Si è deciso di dividere il class diagram in vari diagrammi separati, organizzati per funzionalità, per semplificare la consultazione del documento. Inoltre, abbiamo riportato tutte le classi Model in un unico diagramma dettagliato, ogni volta che saranno coinvolte in associazioni nelle varie funzionalità le classi saranno rappresentate in forma ridotta.

5.2.1 Class Diagram 1- Model

Il seguente class diagram rappresenta per esteso e in modo dettagliato le classi del Model. In seguito verranno riportate solo in forma ridotta.

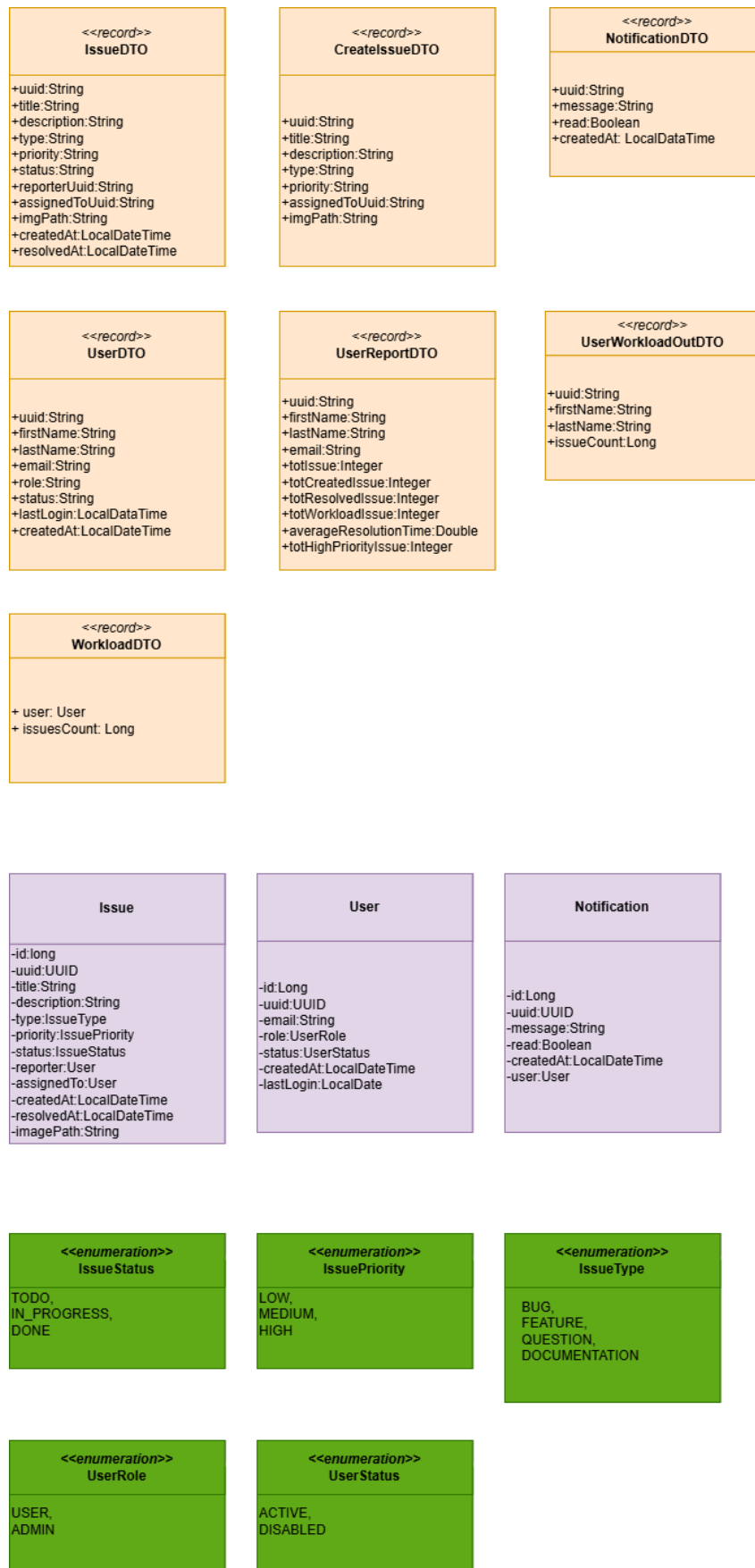


Figura 19: Class Diagram 1 - Model dettagliato

5.2.2 Class Diagram 2- Funzionalità 2

Il seguente class diagram rappresenta le classi coinvolte nella funzionalità 2

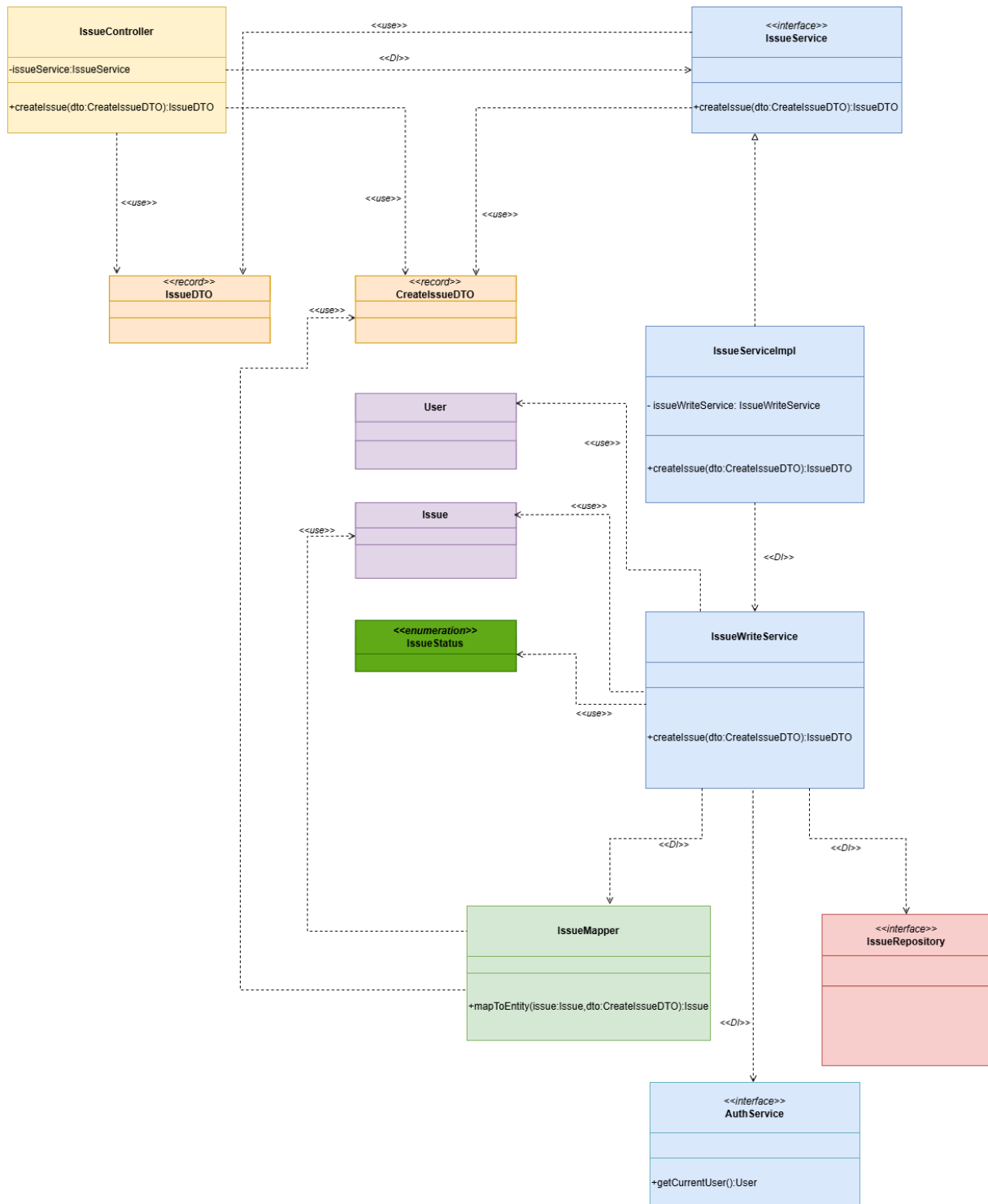


Figura 20: Class Diagram 2 - Funzionalità 2

5.2.3 Class Diagram 3- Funzionalità 3

Il seguente class diagram rappresenta le classi coinvolte nella funzionalità 3

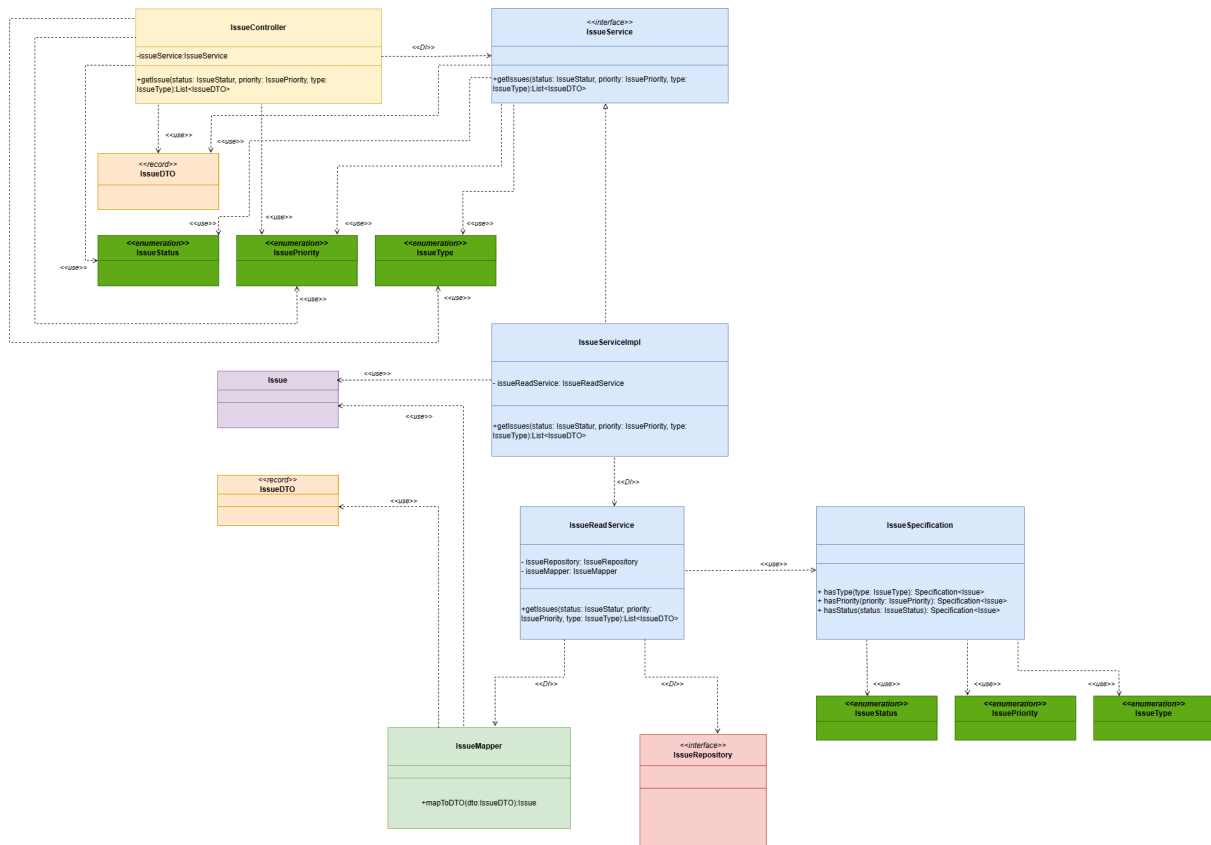


Figura 21: Class Diagram 3 - Funzionalità 3

5.2.4 Class Diagram 4- Funzionalità 4 e 14

Il seguente class diagram rappresenta le classi coinvolte nelle funzionalità 4 e 14

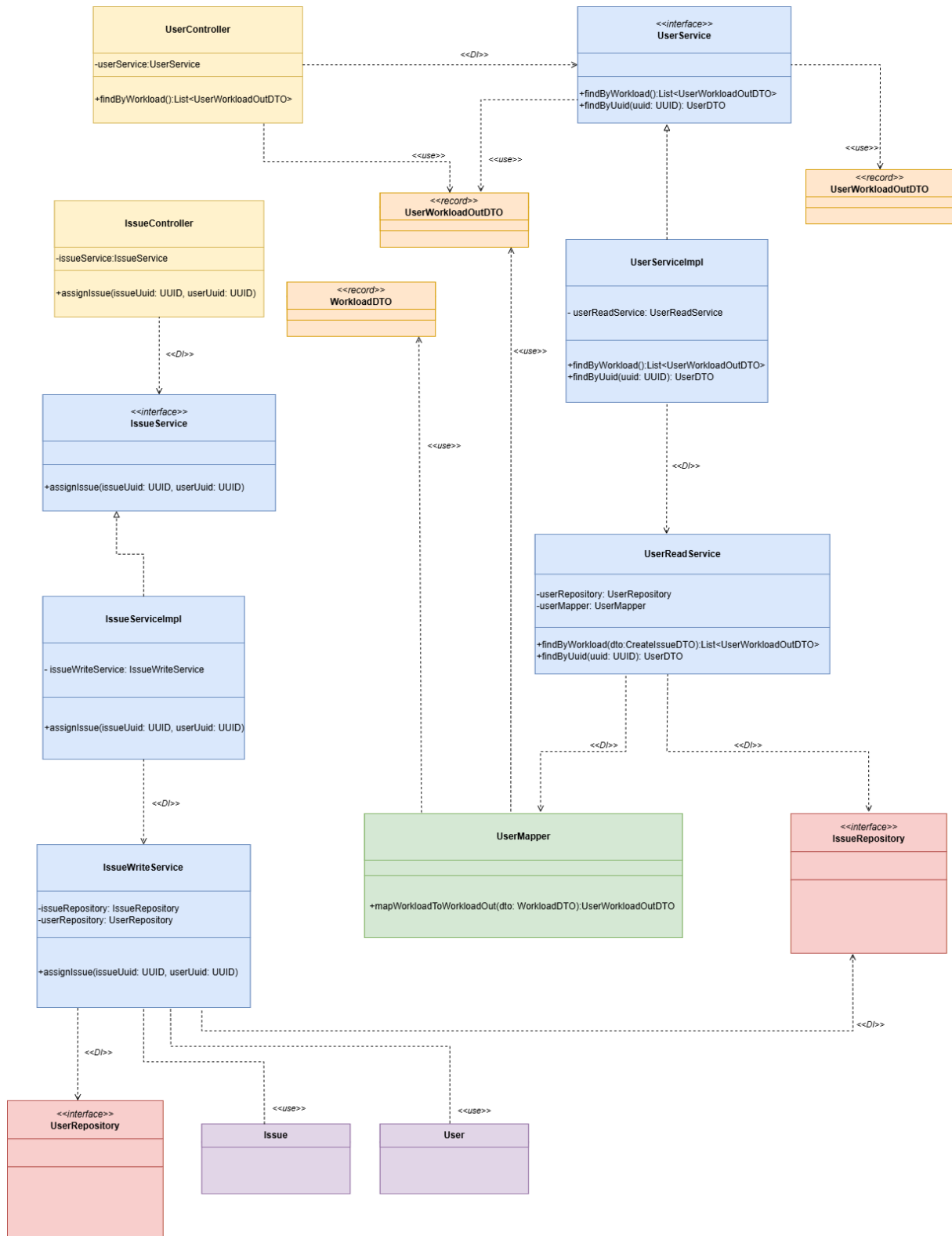


Figura 22: Class Diagram 4 - Funzionalità 4 e 14

5.2.5 Class Diagram 5- Funzionalità 11

Il seguente class diagram rappresenta le classi coinvolte nella funzionalità 11

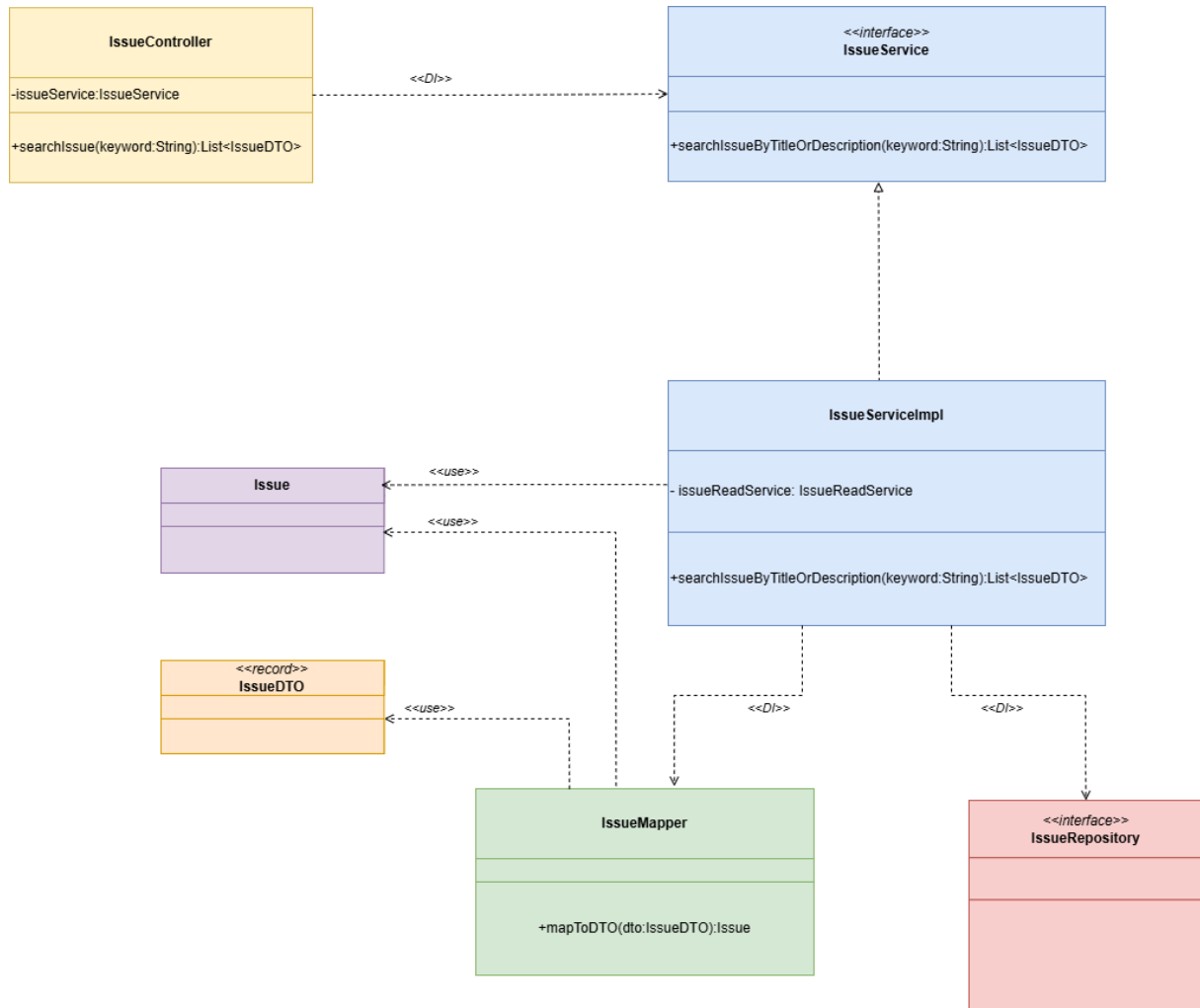


Figura 23: Class Diagram 5 - Funzionalità 11

5.2.6 Class Diagram 6- Funzionalità 17

Il seguente class diagram rappresenta le classi coinvolte nella funzionalità 17

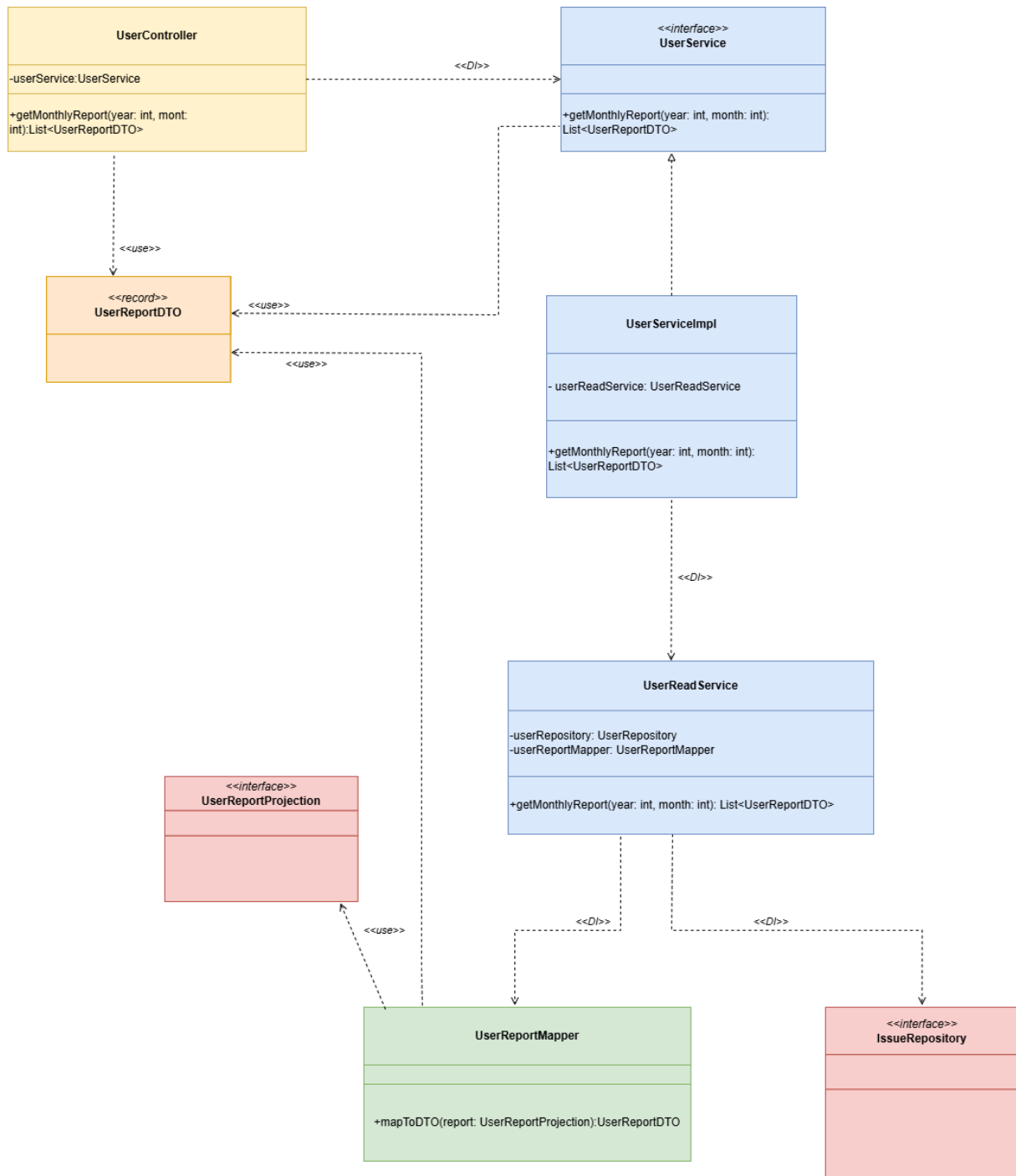


Figura 24: Class Diagram 6 - Funzionalità 17

5.3 Package Diagram Frontend

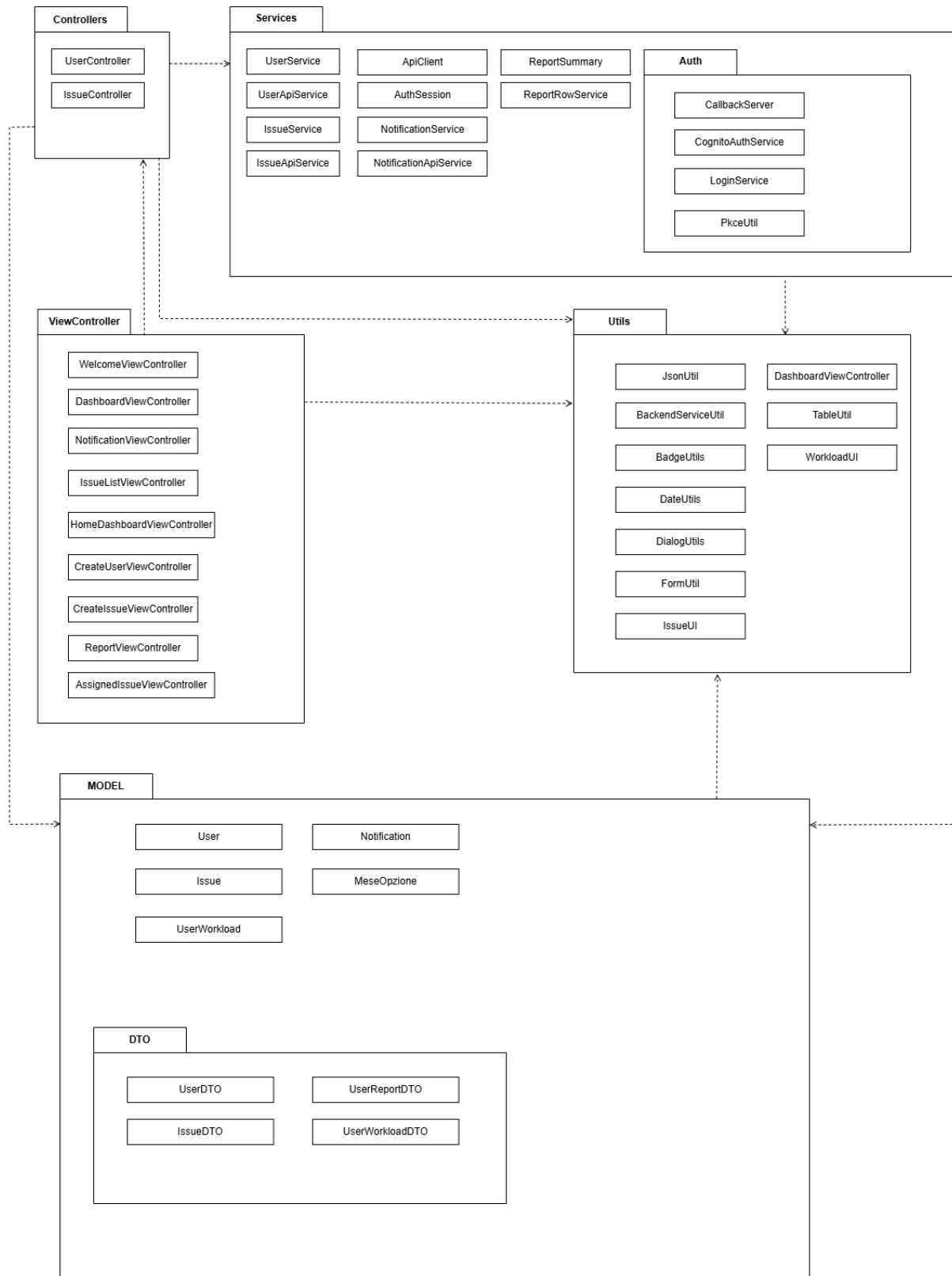


Figura 25: Package Diagram - Frontend

5.4 Descrizione e motivazione delle scelte di software design adottate

Di seguito una lista dei principali design pattern utilizzati nel sistema *BugBoard26*.

- **Dependency injection:** La DI è stata molto utilizzata nello sviluppo del sistema (sia lato Client che lato Server). Questa scelta comporta diversi vantaggi. **Basso accoppiamento:** le classi dipendono da astrazioni (interfacce come `IssueService`, `IssueRepository`) e non da implementazioni concrete, coerentemente con il principio di Dependency Inversion (SOLID). Questo permette di sostituire un'implementazione (es. `IssueServiceImpl`) senza modificare le classi che la utilizzano. **Testabilità:** la possibilità di iniettare mock delle interfacce al posto delle implementazioni reali ha semplificato la scrittura di test di unità (es. testare `IssueServiceImpl` iniettando il mock `IssueRepository`, senza necessità di un database reale). **Bean di Spring:** il ciclo di vita e la creazione delle istanze (bean) sono gestiti da Spring, evitando la duplicazione di logica di istanziazione sparsa nel codice.
- **Singleton:** Il pattern Singleton viene utilizzato per la classe `AuthSession` del *frontend*. Crea una istanza della classe alla quale è possibile accedere dall'esterno per ottenere il token necessario per costruire una corretta chiamata API al *backend*. Naturalmente espone un costruttore privato per evitare la creazione di ulteriori istanze della classe.
- **Event-Listener:** L'*Event-Listener* è un pattern architetturale peculiare di Spring. È stato scelto in sostituzione del design pattern Observer, di cui è una specializzazione in chiave tecnologica Spring. Si occupa della gestione degli eventi, disaccoppiando il componente che genera un evento (*publisher*) da quello che deve gestire le conseguenze (*listener*), senza che il primo debba conoscere direttamente il secondo.

Nel sistema, questo meccanismo è utilizzato per la gestione delle notifiche. Quando un amministratore assegna una *Issue* ad un utente, l'`IssueService` non invoca direttamente il servizio di notifica, ma pubblica un evento (es. `IssueAssignedEvent`) tramite `ApplicationEventPublisher`, contenente i riferimenti necessari (identificativo della issue e dell'utente assegnatario). Un componente listener dedicato (es. `NotificationEventListener`), registrato tramite l'annotazione `@EventListener`, intercetta l'evento e si occupa di creare la relativa notifica nel model e nel database. Questa scelta comporta due importanti vantaggi. In primo luogo genera un **basso accoppiamento**. L'`IssueService` non ha alcuna dipendenza diretta dal servizio di notifica, rispettando il principio SOLID di *Single Responsibility*: la logica di assegnazione della issue resta separata dalla logica di generazione delle notifiche. Inoltre, è possibile aggiungere ulteriori listener per lo stesso evento senza modificare il codice del componente publisher.

Oltre ai Pattern appena mostrati, nel sistema *Bugboard26* sono state prese alcune scelte di Design per rispettare i principi SOLID e per ottenere un **basso accoppiamento e un'al-**

ta coesione. Tra le principali spiccano l'utilizzo di classi **DTO** (Data Transfer Object) utilizzati per disaccoppiare il Model dallo strato di trasferimento dei dati (la View del backend è il serializzatore/deserializzatore di Spring). Inoltre, è usato per mascherare i Model mostrando al richiedente solamente i dati a cui ha effettivamente accesso. Le classi **Mapper** si occupano della conversione tra i Model e i DTO.

Le classi *Services* sono organizzate secondo una gerarchia che parte dall'interfaccia *Service*. Essa ci permette di mantenere il disaccoppiamento tra i *Controller* e le classi *Service* concrete. Oltre a questa gerarchia, si è progettata una struttura divisa per tipologia di richiesta ai *Repositories*: lettura (*read*) e scrittura (*write*). Questa distinzione netta permette un'alta manutenibilità del codice data la semplicità nella localizzazione di eventuali errori.

La classe *IssueSpecification* svolge un ruolo cruciale per la manutenibilità e la scalabilità del codice. Introducendo questo componente, è stato possibile evitare una complessa struttura di *if-else* nidificati per la gestione dei filtri di ricerca, delegando la logica di filtraggio delle issue a un'architettura più snella e facilmente estendibile.

Nel *frontend* non è possibile utilizzare una semplice Dependency Injection perchè i *ViewController* di *JavaFX* non hanno costruttore. Il primo metodo eseguito è il metodo `initialize()`, quindi si è deciso di usare un nuovo metodo `initDependency()` chiamato dal *DashboardViewController* in cui si settano le dipendenze esterne (*Controller* e *Service*), esso verrà eseguito subito dopo l'esecuzione del `initialize()` (nel quale ci sono solamente creazione di componenti grafici, ma non il caricamento dei dati al loro interno).

Per avere un basso accoppiamento e un'alta coesione nelle classi *ViewController* del *frontend* si è spostata la logica di presentazione grafica ad alcune classi di supporto, alle quali i *ViewController* delegano la creazione di tabelle, grafici, badge, ect. Tutti gli stili grafici sono raggruppati in una classe *CSS style.css*.

5.5 Strumento di Versioning

Abbiamo adottato un approccio di versioning basato su **Github**. Per la gestione del versionamento del codice sorgente e della documentazione è stato utilizzato GitHub. È stato adottato un unico repository per l'intero progetto, organizzato nelle seguenti directory principali:

- *Backend/*: codice sorgente del server Spring Boot.
- *Frontend/*: codice sorgente del client *JavaFX*.
- *Documentazione/*: documentazione di progetto (SRS, mockup, diagrammi).

Il repository è raggiungibile al seguente link: <https://github.com/ManuelMontuori/BugBoard26>

La Repository GitHub è composta come segue.

 ManuelMontuori Documentazione	1b15081 · 14 hours ago	 164 Commits
 .vscode	fix test assignissue	3 weeks ago
 Backend/api	Fix backend e aggiunta class diagram exception	5 days ago
 Documentazione	Documentazione	14 hours ago
 Frontend	Documentazione	14 hours ago
 README.md	Initial commit	9 months ago

Figura 26: Repository Github

Il report delle attività del team sono presentate in questa schermata estratta da GitHub.

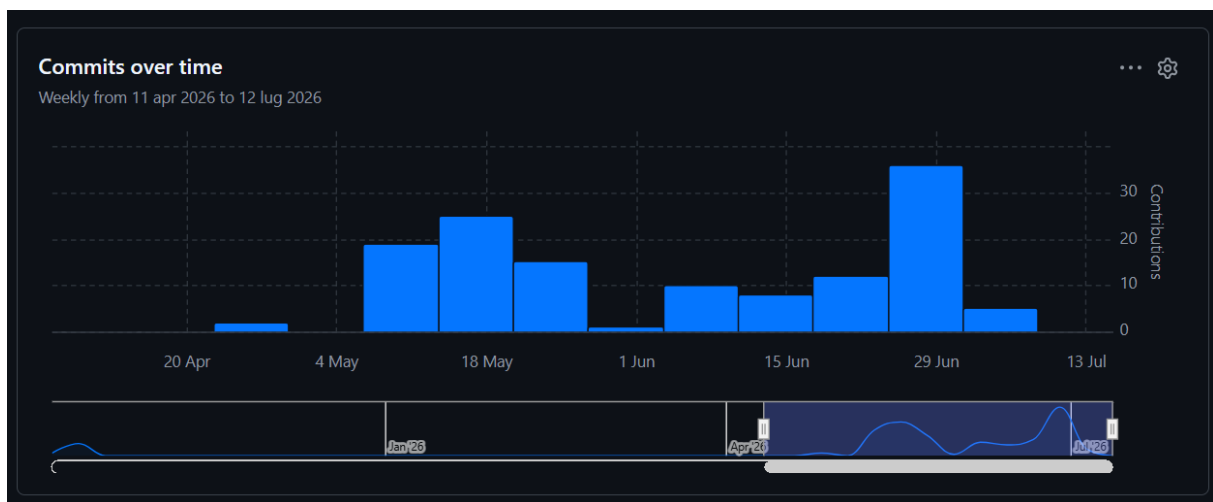


Figura 27: Report Github

5.6 Containerizzazione, build automatica e Cloud Deploy

Il **backend** è stato containerizzato utilizzando *Docker* e caricato su una macchina virtuale AWS (*Amazon Web Services*).

5.6.1 Build automatica

La build del back-end è automatizzata tramite **Maven** (file `pom.xml`), che gestisce la risoluzione delle dipendenze del progetto e la generazione dell'artefatto JAR eseguibile utilizzato dal Dockerfile (`mvn clean package`).

5.6.2 Containerizzazione

Il back-end è distribuito tramite container **Docker**. Il file Dockerfile ([Codice](#)) definisce un'immagine leggera basata su `eclipse-temurin:21-jre-alpine`, che include

la sola JRE (Java Runtime Environment) necessaria all'esecuzione dell'applicazione, già pre-compilata come artefatto JAR tramite la build Maven descritta in precedenza. L'immagine copia il jar risultante dalla build (`target/api-0.0.1-SNAPSHOT.jar`), espone la porta 8080 (porta di default di Spring Boot) e avvia l'applicazione come processo principale del container.

Listing 1: Docker File

```
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY target/api-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Per orchestrare l'esecuzione del container in ambiente di produzione è stato definito un file `docker-compose.yml` ([Codice](#)), che specifica:

- l'immagine da utilizzare, pubblicata su **Amazon ECR** (Elastic Container Registry);
- il mapping tra la porta 80 dell'host e la porta 8080 esposta dal container;
- le variabili d'ambiente necessarie alla configurazione, tra cui l'URL di connessione al database **PostgreSQL** ospitato su **Amazon RDS** (Relational Database Service), e le credenziali di accesso ai servizi AWS;
- la policy di riavvio automatico del container (`restart: always`) in caso di crash o riavvio dell'host.

Listing 2: Docker Compose per il deployment del back-end

```
version: '3.8'
services:
  backend:
    image: <ECR_REGISTRY>/bugboard26:latest
    container_name: bugboard-backend
    ports:
      - "80:8080"
    environment:
      SPRING_DATASOURCE_URL: "jdbc:postgresql://<RDS_ENDPOINT>:5432/postgres"
      SPRING_DATASOURCE_USERNAME: "postgres"
      SPRING_DATASOURCE_PASSWORD: "***"
      AWS_ACCESS_KEY_ID: "***"
      AWS_SECRET_ACCESS_KEY: "***"
      AWS_REGION: "eu-south-1"
```

```
restart: always
```

Questa configurazione consente il deployment del back-end su infrastruttura Cloud (AWS), rendendolo accessibile via rete Internet. Per motivi di sicurezza le credenziali di accesso al database e le chiavi private aws sono state sostituite con ***.

5.6.3 Deploy AWS

Per il Deploy su AWS sono stati utilizzati i seguenti servizi Amazon:

- **Amazon ECR (Elastic Container Registry):** è stato usato per ospitare le immagini dockerizzate del backend. Il servizio è raggiungibile in un Server situato a Milano.
- **Amazon EC2:** è stato utilizzato per creare una macchina virtuale (basata su Linux) nella quale è stato installato Docker, creato il file *docker-compose.yml* e fatto il pull dell'ultima immagine del backend presente su ECR.
- **Amazon RDS (Relational Database Service):** è stato utilizzato per ospitare il Database del sistema Bugboard26, impostato per essere accessibile tramite internet dal backend.
- **Amazon Cognito:** il servizio Cognito è stato centrale nella progettazione dell'applicativo, tramite esso è stato possibile sviluppare un software con un sistema di autenticazione semplice e sicuro. All'attributo email di Cognito sono stati aggiunti gli attributi custom: uuid e role.

I servizi Amazon ECR e Amazon EC2 sono stati configurati e utilizzati tramite terminale, grazie all'installazione del tool *AmazonCLI*.

5.6.4 Report di qualità del codice

Al fine di valutare la qualità del codice del back-end, è stata condotta un'analisi tramite **SonarQube**. L'analisi ha permesso di individuare potenziali problemi di affidabilità, sicurezza e manutenibilità del codice. Di seguito sono riportati i dati dell'ultima modifica effettuata sul Server.

Indice di duplicazione -



Figura 28: SonarQube Report - Indice di duplicazione

Come illustrato dal grafico, nel progetto non vi sono righe duplicate. L'assenza di codice ripetuto riflette un'architettura software efficiente, che previene l'aumento ingiustificato della complessità e garantisce un'ottima manutenibilità a lungo termine.

Indice degli Errori -



Figura 29: SonarQube Report - Indice degli Errori

Come illustrato dal grafico, all'inizio dello sviluppo sono stati trovati 20 errori. Il nuovo codice ne ha generato altri 15, ma alla fine dello sviluppo sono stati completamente azzerati. Ciò indica un processo di miglioramento continuo della qualità del codice, con un'attenta risoluzione dei problemi rilevati durante l'analisi.

Report di qualità Overall -

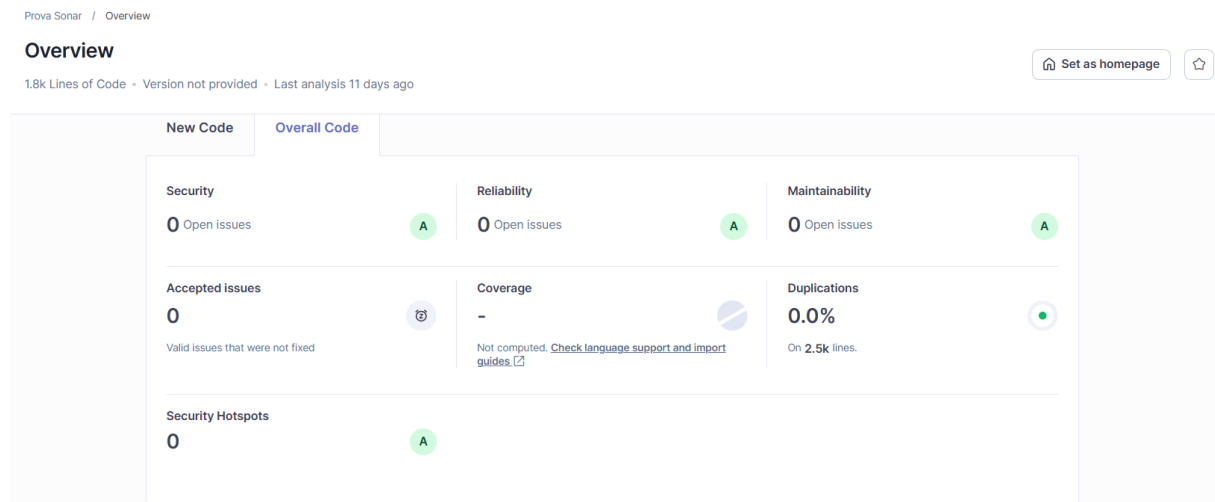


Figura 30: SonarQube Report - Report Overall codice

Report di qualità NewCode -

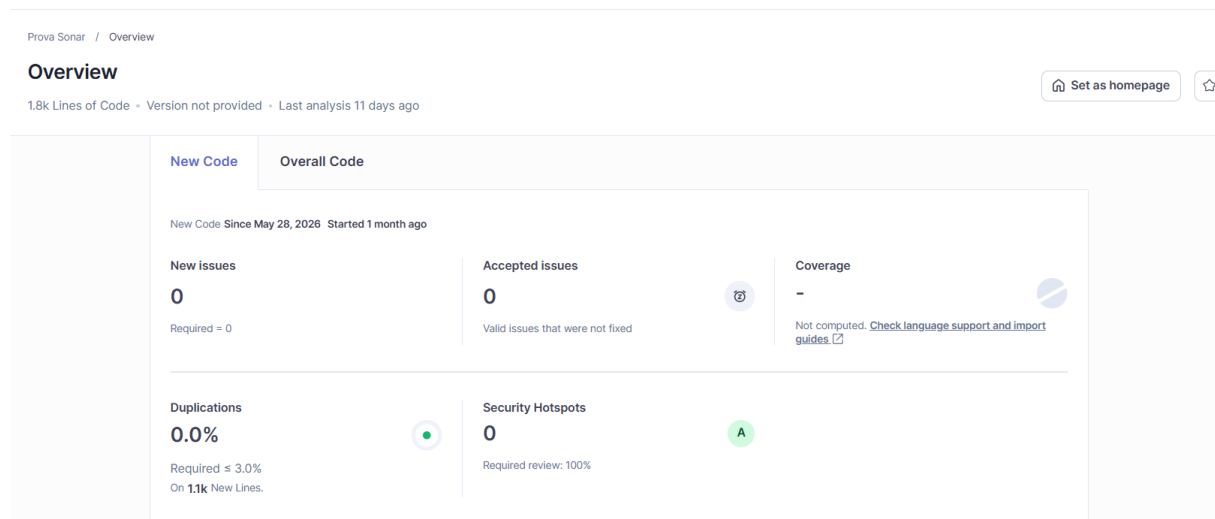


Figura 31: SonarQube Report - Report NewCode

6 Test Plan

6.1 Test della funzionalità RE04 – Assegna issue

Questo Test Plan è riferito alla funzionalità 4 della traccia:

"Ogni bug dovrebbe poter essere assegnato (da un amministratore) a un membro del team. Quando viene assegnato un bug a un utente, quest'ultimo riceve una notifica. Gli utenti possono visualizzare i bug loro assegnati."

Dato il requisito, il test verificherà l'assegnazione di una Issue ad un membro del team e la generazione della relativa notifica.

6.1.1 Analisi delle classi di equivalenza

Input 1 – Issue UUID

Classe valida EC1: Issue esistente

Classe non valida EC2: Issue inesistente

Input 2 – User UUID

Classe valida EC3: Utente esistente

Classe non valida EC4: Utente inesistente

Input 3 – Ruolo utente che effettua l'operazione

Classe valida EC5: Amministratore

Classe non valida EC6: Utente non amministratore

6.1.2 TC1 – Assegnazione corretta

ID	TC1
Descrizione	Assegnazione di una issue esistente ad un utente esistente da parte di un amministratore
Precondizioni	Esiste una Issue; esiste un User; l'utente loggato è amministratore
Input	issueUuid valido, userUuid valido
Output atteso	Operazione completata
Postcondizioni	La Issue è assegnata all'utente; assignedAt valorizzato; status = In Progress; evento pubblicato; notifica creata

6.1.3 TC2 – Utente assegnatario inesistente

ID	TC2
Descrizione	Tentativo di assegnazione ad un utente inesistente
Precondizioni	Esiste una Issue
Input	issueUuid valido, userUuid inesistente
Output atteso	ResourceAccessException
Postcondizioni	Nessuna modifica alla Issue; nessuna notifica

6.1.4 TC3 – Issue inesistente

ID	TC3
Descrizione	Tentativo di assegnazione di una Issue inesistente
Precondizioni	Esiste l'utente destinatario
Input	issueUuid inesistente, userUuid valido
Output atteso	ResourceAccessException
Postcondizioni	Nessuna assegnazione; nessuna notifica

6.1.5 TC4 – Issue inesistente e utente inesistente

ID	TC4
Descrizione	Entrambi gli input non sono validi
Precondizioni	Nessuna
Input	issueUuid inesistente, userUuid inesistente
Output atteso	ResourceAccessException
Postcondizioni	Nessuna modifica al sistema

6.1.6 TC5 – Verifica creazione notifica

ID	TC5
Descrizione	Verifica che venga generata una notifica dopo l'assegnazione
Precondizioni	TC1 eseguito con successo
Input	Issue valida, utente valido
Output atteso	Notifica restituita correttamente
Postcondizioni	Nel database è presente una nuova notifica associata all'utente

6.1.7 TC6 – Visualizzazione dei bug assegnati

ID	TC6
Descrizione	L'utente consulta i bug assegnati
Precondizioni	Esiste almeno una Issue assegnata all'utente
Input	Identificativo utente
Output atteso	Lista delle Issue assegnate
Postcondizioni	Nessuna modifica dei dati

6.1.8 TC7 – Assegnazione tentata da utente non amministratore

ID	TC7
Descrizione	Un utente non amministratore tenta di assegnare una Issue
Precondizioni	Esiste una Issue; esiste un User; l'utente loggato non è amministratore
Input	issueUuid valido, userUuid valido, ruolo = non amministratore
Output atteso	Operazione rifiutata (eccezione di autorizzazione)
Postcondizioni	Nessuna modifica alla Issue; nessuna notifica

6.1.9 Copertura delle classi di equivalenza

Classe	Coperta da
EC1 Issue valida	TC1, TC2, TC5
EC2 Issue non valida	TC3, TC4
EC3 Utente valido	TC1, TC3, TC5, TC6
EC4 Utente non valido	TC2, TC4
EC5 Amministratore	TC1, TC5
EC6 Non amministratore	TC7

6.2 Test del metodo `assignIssue` – `IssueWriteService`

6.2.1 Metodo sottoposto a test

È stato scelto il metodo `assignIssue(UUID issueUuid, UUID userUuid)` della classe `IssueWriteService`. Il metodo ha il compito di assegnare una issue ad un utente.

Il comportamento previsto è:

1. Recuperare l'issue tramite `issueUuid`
2. Recuperare l'utente tramite `userUuid`
3. Associare l'utente all'issue
4. Aggiornare lo stato dell'issue e la data di assegnazione
5. Pubblicare un evento di assegnazione

6.2.2 Strategia adottata

Per il testing è stato adottato un approccio di **unit testing** con Mockito. Sono stati utilizzati mock per simulare il comportamento delle seguenti classi: `IssueRepository`, `UserRepository`, `AuthService`, `IssueMapper`, `ApplicationEventPublisher`.

6.2.3 Black box

Il Black box testing è basato solo su input/output. Sono state utilizzate le tecniche **Equivalence Class Partitioning (ECP)** e **Each Choice Coverage**.

Parametro issueUuid

Classe	Descrizione	Valore rappresentativo
CE1	issue esistente	UUID valido presente nel repository
CE2	issue inesistente	UUID valido non presente nel repository

Parametro userUuid

Classe	Descrizione	Valore rappresentativo
CE3	utente esistente	UUID valido presente nel repository
CE4	utente inesistente	UUID valido non presente nel repository

6.2.4 Casi di test progettati

TC-AI1 – Assegnazione corretta

Input	issueUuid valido, userUuid valido
Precondizioni	Il repository issue restituisce un'issue; il repository utenti restituisce un utente
Output atteso (Oracle)	L'issue viene assegnata all'utente; il campo <code>assignedAt</code> viene valorizzato; lo stato della issue passa a <code>IN_PROGRESS</code> ; viene pubblicato un evento <code>IssueAssignedEvent</code>
Test JUnit	<code>testAssignIssueValidUser()</code>

TC-AI2 – Utente non trovato

Input	issueUuid valido, userUuid inesistente
Precondizioni	L'issue esiste; l'utente non viene trovato
Output atteso (Oracle)	Il metodo deve lanciare <code>ResourceAccessException</code>
Test JUnit	<code>testAssignIssueUserNotFound()</code>

TC-AI3 – Issue non trovata

Input	issueUuid inesistente, userUuid valido
-------	--

Precondizioni	Il repository issue restituisce <code>Optional.empty()</code>
Output atteso (Oracle)	Il metodo deve lanciare <code>ResourceAccessException</code>
Test JUnit	<code>testAssignIssueIssueNotFound()</code>

6.2.5 White box

Analizzando il codice sorgente di `assignIssue`, i soli punti di decisione (branch) presenti sono i due controlli `orElseThrow` su `issueRepository.findById(...)` e `userRepository.findById(...)`; il resto del metodo è codice sequenziale privo di condizioni.

Ramo	Coperto
Issue trovata (TC-AI1, TC-AI2)	✓
Issue non trovata (TC-AI3)	✓
Utente trovato (TC-AI1)	✓
Utente non trovato (TC-AI2)	✓

I 3 casi di test esistenti raggiungono quindi la copertura completa dei branch effettivi del metodo (100% branch coverage).

Nota: `AuthService` e `IssueMapper` sono dipendenze del costruttore di `IssueWriteService` (usate dal metodo `createIssue`, non documentato in questa sezione), ma **non vengono mai richiamate all'interno di `assignIssue`**. Sono mockate solo perché necessarie per istanziare la classe tramite `@InjectMocks`, non perché il metodo sotto test le utilizzi: non rappresentano quindi rami di codice non coperti, semplicemente non sono pertinenti a questo metodo. Si segnala inoltre che, a livello di codice, `assignIssue` non effettua alcun controllo sul ruolo di chi esegue l'operazione: se il requisito "l'assegnazione deve poter essere effettuata solo da un amministratore" è verificato, ciò avviene altrove (es. a livello di controller/endpoint), non in questo metodo.

6.2.6 Oracle dei test

Per determinare la correttezza del risultato sono controllati lo stato interno dell'oggetto `issue`, il lancio di eccezioni e la pubblicazione dell'evento.

Controllo del comportamento in caso di successo:

```
assertEquals(user, issue.getAssignedTo());

assertNotNull(issue.getAssignedAt());

assertEquals(
    IssueStatus.IN_PROGRESS,
```

```

        issue.getStatus()
    );

```

Controllo del comportamento in caso di errore:

```

assertThrows(
    ResourceAccessException.class,
    () -> ...
);

```

Controllo della pubblicazione dell'evento:

```

verify(eventPublisher)
    .publishEvent(any(IssueAssignedEvent.class));

```

6.3 Test del metodo `createNotification` – `NotificationWriteService`

6.3.1 Metodo sottoposto a test

È stato scelto il metodo `createNotification(Issue issue, User user)` della classe `NotificationWriteService`. Il metodo ha il compito di creare una notifica associata ad una issue e ad un utente, aggiungerla al database tramite repository e convertirla in DTO.

Il comportamento previsto è:

1. Creare un oggetto `Notification`
2. Impostare l'utente destinatario
3. Generare il messaggio della notifica a partire dai dati della Issue (titolo e descrizione)
4. Impostare lo stato iniziale della notifica (non letta)
5. Salvare la notifica tramite repository
6. Convertire la notifica salvata in DTO

6.3.2 Strategia adottata

Per il testing è stato adottato un approccio di **unit testing** con Mockito. Sono stati utilizzati mock per simulare il comportamento delle seguenti classi: `NotificationRepository`, `NotificationMapper`.

6.3.3 Black box

Il Black box testing è basato sull'analisi input/output. Sono state utilizzate le tecniche **Equivalence Class Partitioning (ECP)** e **Each Choice Coverage**.

Parametro Issue

Classe	Descrizione	Valore rappresentativo
CE1	Issue valida con titolo e descrizione presenti	Issue completa e corretta

Parametro User

Classe	Descrizione	Valore rappresentativo
CE2	User valido esistente nel sistema	User presente e valido

Persistenza

Classe	Descrizione	Valore rappresentativo
CE3	Salvataggio corretto della notifica	Repository funzionante
CE4	Fallimento del salvataggio	<code>RuntimeException</code> dal repository

6.3.4 Casi di test progettati**TC-CN1 – Creazione notifica corretta**

Input	Issue valida (con titolo e descrizione), User valido
Precondizioni	Il repository salva correttamente la notifica; il mapper converte correttamente l'entità salvata
Output atteso (Oracle)	La notifica viene creata correttamente; l'utente viene associato alla notifica; il campo <code>read</code> è impostato a <code>false</code> ; il messaggio viene generato correttamente; viene restituito il DTO corretto
Test JUnit	<code>testCreateNotificationValid()</code>

TC-CN2 – Errore del repository

Input	Issue valida, User valido, repository che genera eccezione
Precondizioni	Il salvataggio della notifica fallisce
Output atteso (Oracle)	Il metodo deve lanciare <code>RuntimeException</code> ; il mapper non deve essere invocato
Test JUnit	<code>testCreateNotificationWhenRepositoryThrowsException()</code>

6.3.5 Oracle dei test

Per determinare la correttezza del risultato sono stati controllati lo stato interno dell'oggetto creato, il risultato finale del metodo e l'interazione con Repository e Mapper.

Controllo del comportamento in caso di successo:

```
assertEquals(user, captured.getUser());

assertFalse(captured.isRead());

assertEquals(
    "Ti_è_stata_assegnata_la_seguente_Issue:_Errore_login\nDescrizione
    :L'utente_non_riesce_ad_accedere.",
    captured.getMessage()
);
```

Controllo del comportamento in caso di errore:

```
assertThrows(
    RuntimeException.class,
    () -> service.createNotification(issue, user)
);
```

Controllo delle interazioni con Repository e Mapper: Il test reale utilizza un `ArgumentCaptor` per catturare l'oggetto `Notification` istanziato internamente dal metodo, anziché un semplice `any()`:

```
verify(notificationRepository)
    .save(notificationCaptor.capture());

Notification captured = notificationCaptor.getValue();

verify(notificationMapper)
    .mapToDTO(savedNotification);

verify(notificationMapper, never())
    .mapToDTO(any());
```

6.4 Test del metodo `getMonthlyReport` – `UserReadService`

6.4.1 Metodo sottoposto a test

È stato scelto il metodo `getMonthlyReport(int year, int month)` della classe `UserReadService`. Il metodo ha il compito di recuperare i dati relativi agli utenti per un determinato mese e anno, costruendo un report mensile.

Il comportamento previsto è:

1. Calcolare il range temporale del mese tramite `LocalDateTime.of(year, month, 1, 0, 0)` (`startDate`) e `startDate.plusMonths(1)` (`endDate`)
2. Recuperare i dati dal repository nel range temporale
3. Mappare il risultato in DTO
4. Restituire una lista di `UserReportDTO`

Nota: il metodo non esegue alcuna validazione esplicita di `year/month`. Il lancio di `DateTimeException` per mesi non validi (0 o 13) è un effetto collaterale del costruire `LocalDateTime.of(...)` della libreria standard Java, non di un controllo scritto nel metodo.

6.4.2 Strategia adottata

Per il testing è stato adottato un approccio di **unit testing** con Mockito. Sono stati utilizzati mock per simulare il comportamento delle seguenti classi: `UserRepository`, `UserReportMapper`.

6.4.3 Black box

Il Black box testing è basato sull'analisi input/output. Sono state utilizzate le tecniche **Equivalence Class Partitioning (ECP)**, **Boundary Value Analysis (BVA)** e **Each Choice Coverage**.

Parametro year

Classe	Descrizione
CE1	anno valido (intero positivo, es. 2024)

Parametro month Il dominio del parametro è $month \in [1, 12]$.

Classe	Descrizione
CE3	mese valido (1–12)
CE4	mese non valido inferiore al range (< 1)
CE5	mese non valido superiore al range (> 12)

Boundary Value Analysis (BVA)

Tipo	Valore	Significato
BVA1	1	limite inferiore valido
BVA2	12	limite superiore valido

BVA3	0	primo valore non valido
BVA4	13	primo valore oltre il limite

Stato dei dati nel repository

Classe	Descrizione
CE6	presenza di dati per il mese richiesto
CE7	assenza di dati (lista vuota)

6.4.4 Casi di test progettati

TC-GMR1 – Report mensile con dati presenti

Input	year = 2024, month = 5
Precondizioni	Il repository restituisce almeno una <code>UserReportProjection</code> ; il mapper converte correttamente le projection
Output atteso (Oracle)	Viene restituita una lista non vuota; il DTO è correttamente mappato; il repository viene chiamato con il range corretto; il mapper viene invocato
Test JUnit	<code>testGetMonthlyReportValid()</code>

TC-GMR2 – Nessun dato disponibile

Input	year = 2024, month = 5
Precondizioni	Il repository restituisce lista vuota
Output atteso (Oracle)	Il metodo restituisce lista vuota; il repository viene comunque chiamato correttamente; il mapper non viene mai invocato; nessuna eccezione viene lanciata
Test JUnit	<code>testGetMonthlyReportEmptyList()</code>

TC-GMR3 – Mese non valido, limite superiore (BVA4)

Input	year = 2024, month = 13
Output atteso (Oracle)	Il metodo deve lanciare <code>DateTimeException</code>
Test JUnit	<code>testGetMonthlyReportInvalidMonthUpperBound()</code>

TC-GMR4 – Mese non valido, limite inferiore (BVA3)

Input	year = 2024, month = 0
-------	------------------------

Output atteso (Oracle)	Il metodo deve lanciare <code>DateTimeException</code>
Test JUnit	<code>testGetMonthlyReportInvalidMonthLowerBound()</code>

TC-GMR5 – Mese al limite inferiore valido (BVA1)

Input	year = 2024, month = 1
Precondizioni	Il repository restituisce lista vuota
Output atteso (Oracle)	Il metodo non lancia eccezioni; viene restituita una lista vuota; il repository viene chiamato con il range corretto (1 gennaio – 1 febbraio 2024)
Test JUnit	<code>testGetMonthlyReportBoundaryMinMonth()</code>

TC-GMR6 – Mese al limite superiore valido (BVA2)

Input	year = 2024, month = 12
Precondizioni	Il repository restituisce lista vuota
Output atteso (Oracle)	Il metodo non lancia eccezioni; viene restituita una lista vuota; il repository viene chiamato con il range corretto (1 dicembre 2024 – 1 gennaio 2025)
Test JUnit	<code>testGetMonthlyReportBoundaryMaxMonth()</code>

6.4.5 Oracle dei test

Per la verifica della correttezza sono stati utilizzati: la verifica dello stato del risultato, la verifica delle interazioni con repository e mapper, la verifica delle eccezioni in caso di input non valido.

Controllo del comportamento in caso di successo:

```
assertEquals(1, result.size());
assertEquals(dto, result.get(0));
```

Controllo del caso lista vuota:

```
assertTrue(result.isEmpty());

verify(userReportMapper, never())
    .mapToDTO(any());
```

Controllo del caso input non valido: Lo stesso pattern è applicato sia al limite superiore (month = 13) sia al limite inferiore (month = 0):

```
assertThrows(
    DateTimeException.class,
    () -> userService.getMonthlyReport(2024, 13)
```

```
);
```

Controllo delle interazioni con le dipendenze:

```
verify(userRepository)
    .getUserReports(startDate, endDate);

verify(userReportMapper)
    .mapToDTO(projection);
```

Indice analitico

Amazon Web Services (AWS), [54–56](#)

 Cognito, [42, 56](#)

 EC2, [56](#)

 ECR, [55, 56](#)

 RDS, [55](#)

Assegna, [12](#)

Autenticazione, [6, 12, 14](#)

Class Diagram, [44](#)

Client-Server, [40](#)

Cockburn, [9, 29](#)

Database, [41, 42, 44, 52, 55, 56](#)

Dependency Injection, [52, 53](#)

Docker, [42, 54, 56](#)

DTO, [40, 53](#)

Event-Listener / Observer, [52](#)

Funzionalità, [6, 12](#)

GitHub, [53](#)

Glossario, [7](#)

IEEE Std 830-1998, [9](#)

Issue, [12](#)

 Assegna, [30](#)

 Assegnazione, [16, 30](#)

 Assegnazione issue, [6](#)

 Filtra, [12, 18](#)

 Modifica, [12](#)

 Ordina, [12](#)

 Ordine, [19](#)

 Ricerca, [12, 20](#)

 Segnalazione, [12, 15, 29](#)

JavaFX / FXML, [43, 53](#)

Maven, [54](#)

Mockup, [31, 35](#)

MVC, [40–42](#)

Notifiche, [13](#)

 Invio, [13](#)

 Lettura notifiche, [21](#)

Nuova utenza (Amministratore), [12, 14](#)

PostgreSQL, [43, 55](#)

Principi SOLID, [52](#)

Priorità, [7, 12, 16, 19, 20, 24, 29](#)

Singleton, [52](#)

SonarQube, [56](#)

Spring Boot, [42, 43, 53, 55](#)

Test, [58](#)

 Assegnazione corretta, [59](#)

 Assegnazione da utente non
 amministratore, [61](#)

 assignIssue, [61](#)

 Classi di equivalenza, [59](#)

 Copertura classi di equivalenza, [61](#)

 createNotification, [64](#)

 Creazione notifica, [60](#)

 getMonthlyReport, [66](#)

 Issue e utente inesistenti, [60](#)

 Issue inesistente, [60](#)

 Utente assegnatario inesistente, [59](#)

 Visualizzazione bug assegnati, [60](#)

Use Case, [22](#)

 Assegna issue, [25](#)

 Autenticazione, [23](#)

 Segnala issue, [24](#)

 Visualizza issue assegnate, [25](#)

 Visualizza Notifiche, [27](#)

 Visualizza report team, [26](#)

 Visualizza riepilogo, [26](#)

Vista, [12](#)

Issue personali, [12](#), [18](#)

Notifiche, [12](#), [21](#)

Report mensile, [6](#), [12](#), [18](#)

Riepilogo issue, [6](#), [12](#), [17](#)

Workload, [6](#), [8](#)

Workload, [22](#)